DBMS Experiential Learning – Level 3 Mini Project

PropFind

Semantic Property Search & Rental Management System

Database paradigm: Option 2 – SQL (PostgreSQL) + Vector Database (pgvector)

Web application: Next.js 14 | Source: github.com/jishnu-sivakumar/DBMS_mini

Student name	SRN
KANISHQ SURENDRAN	PES2UG24CS214
JISHNU SIVAKUMAR	PES2UG24CS204

November 2026

Contents

1. Title
2. Students: SRN and Name
3. Abstract
4. Tech Stack
5. SQL – Database Commands
6. Vector Database Commands (NoSQL paradigm)
7. Next.js – Project Tree Structure
8. Sample Screenshots: Application Code and Demo
9. Evaluation Rubrics – Completion Status
10. Appendix: Data Dictionary, Testing, Contributions, Future Work

1. Title

PropFind: Semantic Property Search & Rental Management System

A rental platform where tenants search in natural language (“house where I can keep my dog”, “pet friendly 2BHK in Pune under 25k”), owners list and manage properties, and the full booking life cycle (request, approval, payment, cancellation, audit) is protected by database-level rules.

2. Students: SRN and Name

Name	SRN	GitHub	Responsibility
KANISHQ SURENDRAN	PES2UG24CS214	FLASH860	Authentication and roles, booking life cycle and concurrency control, payments and audit triggers, real embeddings, number-aware search, dataset, owner/tenant dashboards, SQL demonstrations and tests, documentation
JISHNU SIVAKUMAR	PES2UG24CS204	jishnu-sivakumar	Database schema and constraints, pgvector storage and HNSW index, semantic search function, search page, property cards and listing form, initial seed pipeline

3. Abstract

PropFind is a rental-property discovery and management platform built for the DBMS Level 3 mini-project using **Option 2: SQL plus a vector database**. Tenants describe what they want in plain language. Every listing is converted to a 384-dimension embedding with the *all-MiniLM-L6-v2* sentence model and stored in PostgreSQL using the **pgvector** extension. A query is embedded in the same way and the closest listings are retrieved by cosine distance through an **HNSW** index. Because vector similarity cannot compare numbers, constraints written in the sentence (budget, bedrooms, city) are parsed and enforced as exact SQL filters, so "homes under rent 20k" returns only homes that really cost under ₹20,000.

The same PostgreSQL database holds the relational core: users with roles (owner, tenant, admin), properties, bookings, reviews, payments and a booking audit log. Data integrity is enforced inside the database with ENUM types, CHECK / UNIQUE / FOREIGN KEY constraints and seven triggers (double-booking prevention, booking state machine, payment validation and timestamping, audit logging, automatic refund on cancellation and timestamp maintenance). Bookings run as ACID transactions with row-level locking (`SELECT ... FOR UPDATE`).

The Next.js 14 application provides authentication (bcrypt password hashes, JWT in an httpOnly cookie), role-based access control in middleware and in every API route, owner and tenant dashboards with drill-down pages, and a payment flow. The repository includes a 59-listing dataset across 15 cities, labelled SQL demonstrations, and a self-checking SQL test script with 23 checks (all passing) plus 12 parser tests.

Problem statement

Keyword and checkbox filters make it hard to find a home from a vague need ("somewhere quiet where I can keep a pet"), and naive booking code lets two tenants reserve the same dates when requests arrive together. PropFind addresses both: semantic retrieval for discovery, and database-enforced consistency for reservations.

Objectives

- Design a normalised relational schema with strong integrity rules and document it (ER diagram, relational schema, data dictionary).
- Add semantic search with embeddings stored and queried inside PostgreSQL, combined with exact relational filters.
- Prevent double booking and invalid state changes at the database level, independent of application code.
- Provide a working Next.js interface with authentication and two or more roles.

Scope and functional requirements

- Register / log in as owner or tenant; owners publish listings (embedding generated in the same transaction).
- Search in natural language with optional filters (city, bedrooms, budget); see what the system understood.
- Tenants request, cancel and pay for bookings; owners approve or reject; every status change is logged.

- Owner dashboard: properties, pending requests, confirmed bookings, revenue (collected vs outstanding).
- Out of scope: online payment gateway (payment is recorded, not charged), an admin management UI, image upload.

4. Tech Stack

Layer	Technology	Role in PropFind
Frontend	Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, lucide-react	Pages, forms, dashboards, search UI
Backend / API	Next.js Route Handlers (Node.js 24)	REST endpoints, validation, role checks
SQL database	PostgreSQL 16.15	Relational data, constraints, views, functions, triggers, transactions
Vector database	pgvector 0.8.7 (HNSW index, cosine distance)	Embedding storage and similarity search inside PostgreSQL
Embedding model	all-MiniLM-L6-v2 via @huggingface/transformers (ONNX), 384 dimensions	Turns listings and queries into vectors; runs locally, no API key
DB driver	node-postgres (pg)	Parameterised queries, pooled connections, transactions
Authentication	bcryptjs, jose (JWT, HS256), httpOnly cookie	Password hashing, sessions, role-based access
Infrastructure	Docker Compose (pgvector/pgvector:pg16); optional Supabase / Neon	Local database; hosted option documented
Version control	Git + GitHub (feature branches, pull requests, merge commits)	Individual contributions visible in history

System architecture

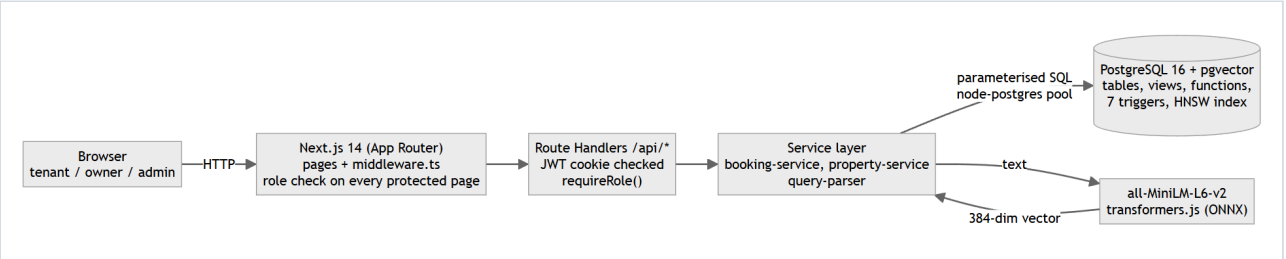


Figure 4.1: System architecture

Configuration (database URL, JWT secret, seed password, optional OpenAI key) comes only from environment variables; `.env` is git-ignored and `.env.example` documents every setting.

5. SQL – Database Commands

5.1 Entity-relationship diagram and relational schema

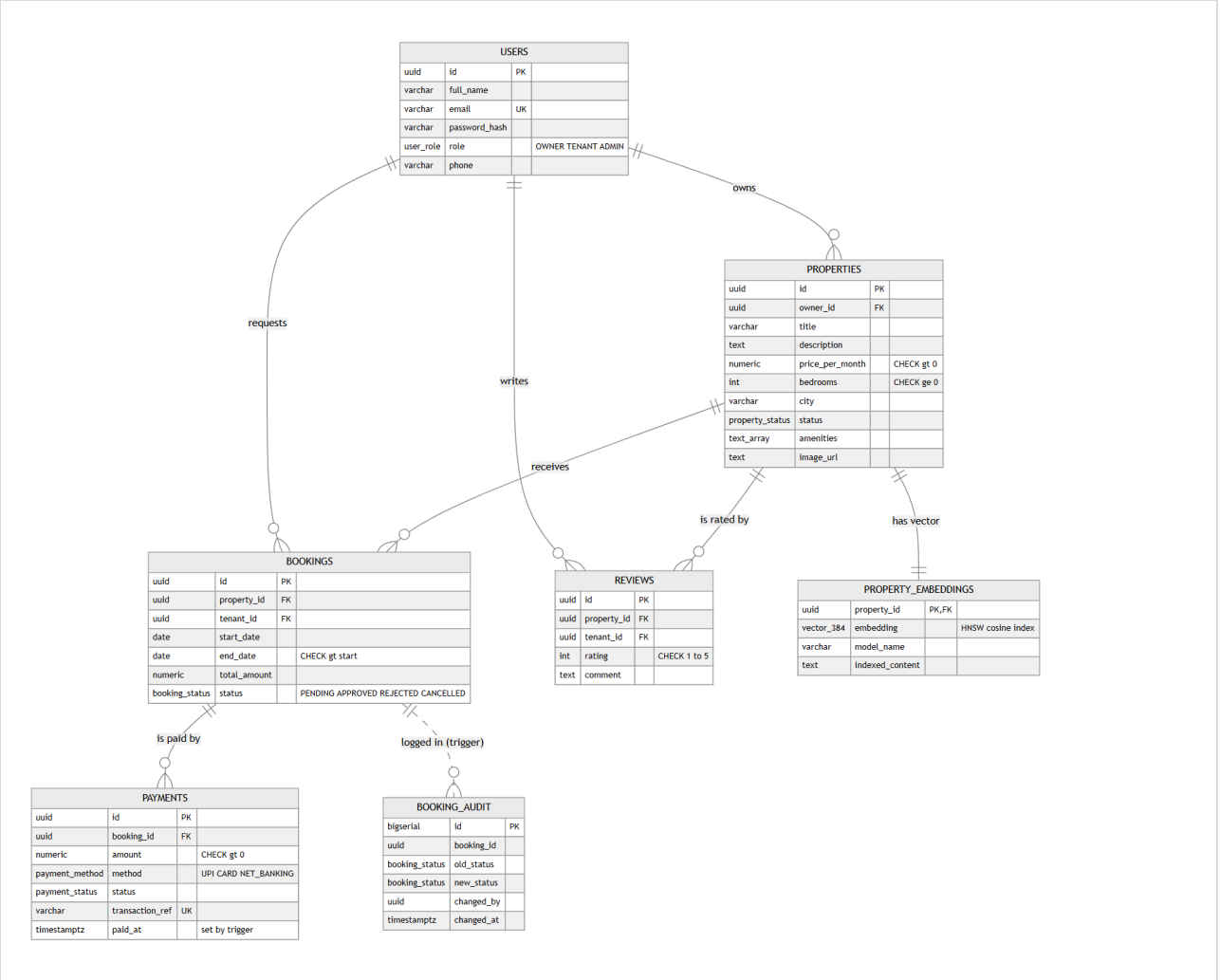


Figure 5.1: ER diagram of the implemented schema (7 tables)

Relational schema (PK = primary key, FK = foreign key, UK = unique):

```
users(id PK, full_name, email UK, password_hash, role, phone, avatar_url, created_at, updated_at)
properties(id PK, owner_id FK→users, title, description, property_type, price_per_month,
security_deposit,
          bedrooms, bathrooms, area_sqft, address, city, state, pincode, status, amenities[],
image_url, created_at, updated_at)
property_embeddings(property_id PK/FK→properties, embedding vector(384), model_name, indexed_content,
created_at, updated_at)
bookings(id PK, property_id FK→properties, tenant_id FK→users, start_date, end_date, total_amount,
status, special_requests, created_at, updated_at)
reviews(id PK, property_id FK→properties, tenant_id FK→users, rating, comment, created_at,
UK(property_id, tenant_id))
payments(id PK, booking_id FK→bookings, amount, method, status, transaction_ref UK, paid_at, created_at)
booking_audit(id PK, booking_id, old_status, new_status, changed_by, changed_at)
```

Relationships: one user (owner) owns many properties; one property receives many bookings and reviews and has exactly one embedding; one user (tenant) makes many bookings and reviews; one booking has

many payments (at most one PAID) and many audit rows. `booking_audit.booking_id` deliberately has no foreign key so the log survives deletions.

5.2 DDL: CREATE (tables, types, extensions)

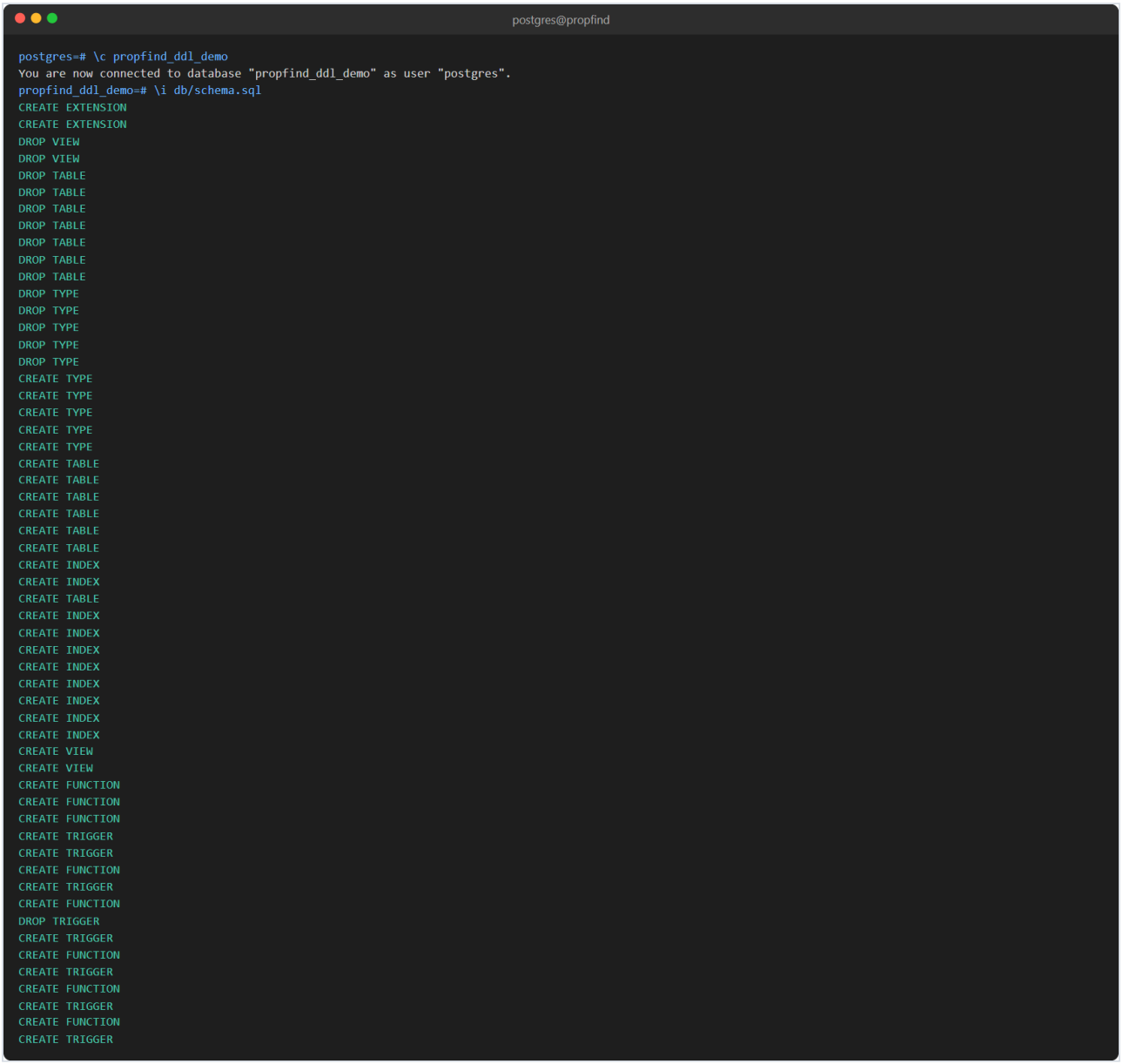


Figure 5.2: Executing db/schema.sql (CREATE EXTENSION, TYPE, TABLE, INDEX, VIEW, FUNCTION, TRIGGER)

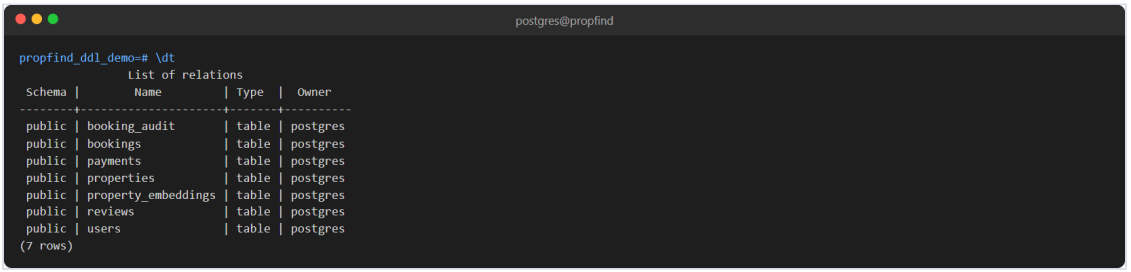


Figure 5.3: \dt shows the 7 tables

db/schema.sql (users and properties tables with CHECK constraints)

```sql
-- 4. Users Table
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    full_name VARCHAR(120) NOT NULL,
    email VARCHAR(160) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role user_role NOT NULL DEFAULT 'TENANT',
    phone VARCHAR(20),
    avatar_url TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT users_email_format CHECK (email ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'),
    CONSTRAINT users_phone_format CHECK (phone IS NULL OR phone ~ '^\+?[0-9 ]{10,20}$')
);

-- 5. Properties Table (Relational Catalog)
CREATE TABLE properties (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    owner_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(200) NOT NULL,
    description TEXT NOT NULL,
    property_type VARCHAR(50) NOT NULL DEFAULT 'Apartment',
    price_per_month NUMERIC(10, 2) NOT NULL CHECK (price_per_month > 0),
    security_deposit NUMERIC(10, 2) NOT NULL DEFAULT 0.00 CHECK (security_deposit >= 0),
    bedrooms INT NOT NULL CHECK (bedrooms >= 0),
    bathrooms INT NOT NULL CHECK (bathrooms >= 0),
    area_sqft INT NOT NULL CHECK (area_sqft > 0),
    address TEXT NOT NULL,
    city VARCHAR(80) NOT NULL,
    state VARCHAR(80) NOT NULL,
    pincode VARCHAR(20) NOT NULL,
    status property_status NOT NULL DEFAULT 'AVAILABLE',
    amenities TEXT[] DEFAULT '{}',
    image_url TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Figure 5.4: users and properties tables with CHECK constraints (db/schema.sql)

postgres@propfind

```
propfind_db=# \d users
                                Table "public.users"
    Column     |           Type           | Collation | Nullable |       Default
---------------+--------------------------+-----------+----------+---------------------
 id            | uuid                     |           | not null | uuid_generate_v4()
 full_name     | character varying(120)   |           | not null |
 email         | character varying(160)   |           | not null |
 password_hash | character varying(255)   |           | not null |
 role          | user_role                |           | not null | 'TENANT'::user_role
 phone         | character varying(20)    |           |          |
 avatar_url    | text                     |           |          |
 created_at    | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
 updated_at    | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "users_pkey" PRIMARY KEY, btree (id)
    "users_email_key" UNIQUE CONSTRAINT, btree (email)
Check constraints:
    "users_email_format" CHECK (email::text ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'::text)
    "users_phone_format" CHECK (phone IS NULL OR phone::text ~ '^\+?[0-9 ]{10,20}$'::text)
Referenced by:
    TABLE "bookings" CONSTRAINT "bookings_tenant_id_fkey" FOREIGN KEY (tenant_id) REFERENCES users(id) ON DELETE RESTRICT
    TABLE "properties" CONSTRAINT "properties_owner_id_fkey" FOREIGN KEY (owner_id) REFERENCES users(id) ON DELETE CASCADE
    TABLE "reviews" CONSTRAINT "reviews_tenant_id_fkey" FOREIGN KEY (tenant_id) REFERENCES users(id) ON DELETE CASCADE
```

Figure 5.5: \d users: primary key, unique and CHECK constraints

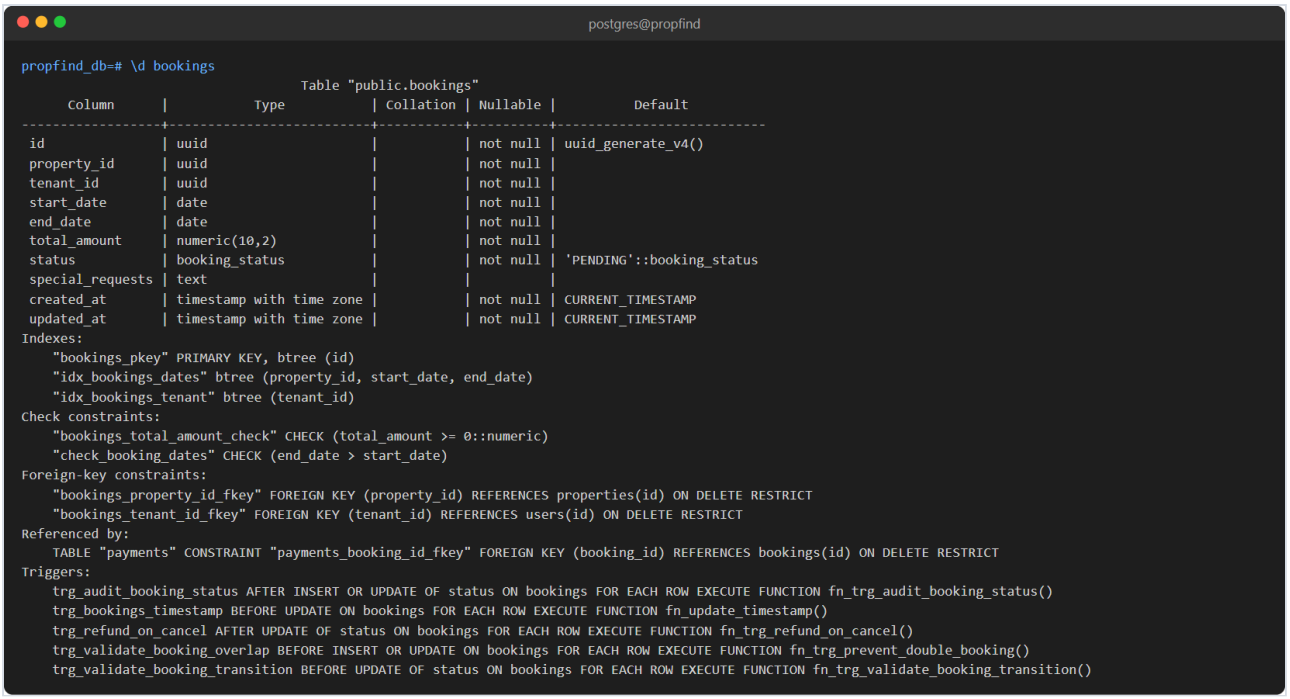


Figure 5.6: \d bookings: foreign keys, CHECK and triggers

5.3 DDL: ALTER and DROP

```
-- L1. ALTER TABLE: add a column, add a CHECK constraint, rename, set NOT NULL, drop
BEGIN;
ALTER TABLE properties ADD COLUMN furnishing VARCHAR(20) DEFAULT 'UNFURNISHED';
ALTER TABLE properties
    ADD CONSTRAINT properties_furnishing_check
    CHECK (furnishing IN ('UNFURNISHED', 'SEMI_FURNISHED', 'FULLY_FURNISHED'));
UPDATE properties SET furnishing = 'FULLY_FURNISHED' WHERE 'Fully Furnished' = ANY(amenities);
UPDATE properties SET furnishing = 'SEMI_FURNISHED' WHERE 'Semi-Furnished' = ANY(amenities);
SELECT furnishing, COUNT(*) AS properties FROM properties GROUP BY furnishing ORDER BY properties DESC;
ALTER TABLE properties RENAME COLUMN furnishing TO furnishing_level;
ALTER TABLE properties ALTER COLUMN furnishing_level SET NOT NULL;
SAVEPOINT before_bad_value;
\set ON_ERROR_STOP off
UPDATE properties SET furnishing_level = 'PARTLY' WHERE id = (SELECT id FROM properties LIMIT 1);  --
violates the new CHECK, error expected
\set ON_ERROR_STOP on
ROLLBACK TO SAVEPOINT before_bad_value;
ALTER TABLE properties DROP COLUMN furnishing_level;   -- also drops its CHECK constraint
ROLLBACK;
SELECT COUNT(*) AS furnishing_columns_left
FROM information_schema.columns WHERE table_name = 'properties' AND column_name LIKE 'furnishing%';
```

```
-- L2. CREATE VIEW / DROP VIEW / DROP TABLE (rolled back)
BEGIN;
CREATE VIEW view_budget_listings AS
    SELECT id, title, city, price_per_month FROM properties WHERE price_per_month < 20000;
SELECT COUNT(*) AS budget_listings, MIN(price_per_month) AS cheapest, MAX(price_per_month) AS dearest
FROM view_budget_listings;
DROP VIEW view_budget_listings;
CREATE TABLE demo_scratch (id INT PRIMARY KEY, note TEXT);
INSERT INTO demo_scratch VALUES (1, 'temporary');
DROP TABLE demo_scratch;
ROLLBACK;
SELECT to_regclass('view_budget_listings') AS view_exists, to_regclass('demo_scratch') AS table_exists;
```

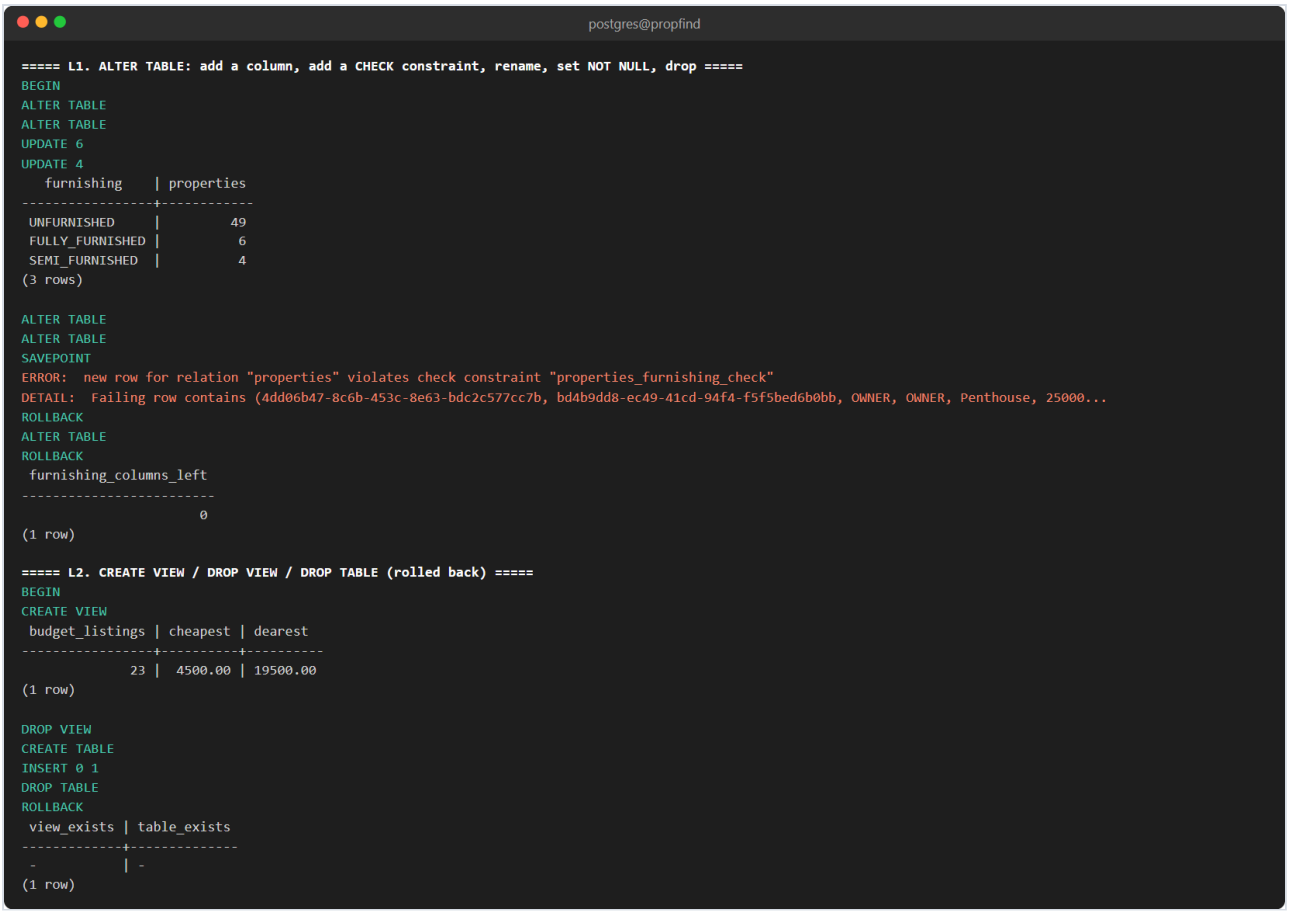


Figure 5.7: ALTER TABLE, CREATE VIEW, DROP VIEW, DROP TABLE (inside a transaction, rolled back)

5.4 DML and queries

INSERT is used by the seed script and the application (all values passed as bind parameters); UPDATE and DELETE are shown below inside rolled-back transactions.

```
-- A1. INNER JOIN (4 tables): every booking with property, tenant and owner
SELECT b.start_date, b.end_date, b.status,
       p.title        AS property,
       t.full_name    AS tenant,
       o.full_name    AS owner
FROM bookings b
JOIN properties p ON p.id = b.property_id
JOIN users t      ON t.id = b.tenant_id
JOIN users o      ON o.id = p.owner_id
ORDER BY b.start_date;
```

```
-- B2. HAVING: cities with at least 2 listings and average rent above 20,000
SELECT city, COUNT(*) AS properties, ROUND(AVG(price_per_month), 2) AS avg_rent
FROM properties
GROUP BY city
HAVING COUNT(*) >= 2 AND AVG(price_per_month) > 20000
ORDER BY avg_rent DESC;
```

postgres@propfind

```
===== A1. INNER JOIN (4 tables): every booking with property, tenant and owner =====
 start_date |  end_date  |  status   |                          property                          |     tenant     |    owner
------------+------------+-----------+------------------------------------------------------------+----------------+--------------
 2026-11-10 | 2027-02-10 | APPROVED  | Hostel Bed for Students near Anna University               | Rohan Nair     | Vikram Mehta
 2026-11-15 | 2026-12-15 | APPROVED  | Serene 2BHK near Manyata Tech Park with Balcony            | Rohan Nair     | Priya Sharma
 2026-11-20 | 2027-01-20 | REJECTED  | Cozy 2BHK Family Home with Modular Kitchen in Whitefield   | Meera Iyer     | Ananya Roy
 2026-11-25 | 2027-02-25 | APPROVED  | Budget 1BHK in Kakkanad IT Corridor                        | Aditi Menon    | Vikram Mehta
 2026-12-01 | 2027-03-01 | REJECTED  | Heritage Bungalow in Civil Lines                           | Sneha Patel    | Arjun Reddy
 2026-12-01 | 2027-01-01 | PENDING   | Budget 1BHK Student Apartment Near Indiranagar Metro       | Sneha Patel    | Rahul Verma
 2026-12-01 | 2027-03-01 | APPROVED  | Modern Fully-Furnished Studio Apartment in Hitec City      | Aditi Menon    | Ananya Roy
 2026-12-05 | 2027-02-05 | CANCELLED | Ultra Luxury 3BHK Penthouse with Private Terrace & Pool Access | Farhan Ali | Priya Sharma
 2026-12-10 | 2027-01-10 | PENDING   | Single Occupancy Student Room near Pune University         | Sneha Patel    | Neha Kapoor
 2026-12-15 | 2027-03-15 | PENDING   | Spacious 2BHK in Banjara Hills                             | Meera Iyer     | Neha Kapoor
 2026-12-20 | 2027-02-20 | APPROVED  | 2BHK Lake-View Apartment on Marine Drive                   | Farhan Ali     | Neha Kapoor
 2027-01-05 | 2027-04-05 | PENDING   | Pet-Friendly Ground Floor 2BHK in Velachery                | Karan Malhotra | Arjun Reddy
 2027-01-10 | 2027-04-10 | PENDING   | Spacious 3BHK Independent Villa with Private Garden        | Karan Malhotra | Rahul Verma
 2027-02-01 | 2027-05-01 | PENDING   | Serviced Apartment in BKC                                  | Rohan Nair     | Neha Kapoor
(14 rows)

===== A2. LEFT JOIN: every property with its booking count (0 if never booked) =====
                          title                           |    city    | bookings
----------------------------------------------------------+------------+----------
 2BHK Lake-View Apartment on Marine Drive                 | Kochi      |        1
 Budget 1BHK in Kakkanad IT Corridor                      | Kochi      |        1
 Budget 1BHK Student Apartment Near Indiranagar Metro     | Bengaluru  |        1
 Cozy 2BHK Family Home with Modular Kitchen in Whitefield | Bengaluru  |        1
 Heritage Bungalow in Civil Lines                         | Jaipur     |        1
 Hostel Bed for Students near Anna University             | Chennai    |        1
 Modern Fully-Furnished Studio Apartment in Hitec City    | Hyderabad  |        1
 Pet-Friendly Ground Floor 2BHK in Velachery              | Chennai    |        1
 Serene 2BHK near Manyata Tech Park with Balcony          | Bengaluru  |        1
 Serviced Apartment in BKC                                | Mumbai     |        1
 Single Occupancy Student Room near Pune University       | Pune       |        1
 Spacious 2BHK in Banjara Hills                           | Hyderabad  |        1
 Spacious 3BHK Independent Villa with Private Garden      | Pune       |        1
 Ultra Luxury 3BHK Penthouse with Private Terrace & Pool Access | Mumbai |        1
 1BHK Garden-Facing Flat in Porvorim                      | Goa        |        0
 1BHK in Gurugram Sector 56 near Rapid Metro              | Gurugram   |        0
 1BHK near Kondapur Metro for IT Professionals            | Hyderabad  |        0
 2BHK near Noida Sector 62 Metro                          | Noida      |        0
 2BHK Sea-Facing Apartment in Panjim                      | Goa        |        0
 2BHK with Balcony near SG Highway                        | Ahmedabad  |        0
 2BHK with Terrace in Koregaon Park                       | Pune       |        0
 3BHK Family Flat in Adyar                                | Chennai    |        0
 3BHK Garden Apartment in Salt Lake                       | Kolkata    |        0
 3BHK Sea-View Apartment in Bandra West                   | Mumbai     |        0
 4BHK Independent Floor in Vasant Vihar                   | Delhi      |        0
 4BHK Independent House in Besant Nagar                   | Chennai    |        0
 4BHK Penthouse in Indiranagar                            | Bengaluru  |        0
 4BHK Sea-Facing Penthouse in Worli                       | Mumbai     |        0
 5BHK Villa with Pool in Jubilee Hills                    | Hyderabad  |        0
 Affordable 2BHK near Salt Lake Sector V                  | Kolkata    |        0
 Budget 2BHK for Families in Wakad                        | Pune       |        0
 Compact Studio Near Sector 17 Plaza                      | Chandigarh |        0
 Co-working Friendly 1BHK in Powai                        | Mumbai     |        0
 Cozy PG Room for Working Women near Koramangala          | Bengaluru  |        0
 Designer 3BHK on Golf Course Extension Road              | Gurugram   |        0
 Duplex 3BHK in Jubilee Hills                             | Hyderabad  |        0
 Farmhouse Retreat near Lonavala                          | Lonavala   |        0
 Fully Furnished 2BHK in Gachibowli                       | Hyderabad  |        0
 Furnished 1BHK near Sarjapur Road IT Hub                 | Bengaluru  |        0
 Gated 3BHK Villa-Style Home in Sarjapur                  | Bengaluru  |        0
 Golf Course Road 2BHK Near Metro                         | Gurugram   |        0
 Heritage 3BHK in Alipore                                 | Kolkata    |        0
 Heritage Style 2BHK Floor in Jaipur Old City             | Jaipur     |        0
 Independent Floor in Sector 21 Chandigarh                | Chandigarh |        0
 Low-cost Bachelor Flat in Thane West                     | Mumbai     |        0
 Luxury Villa with Private Pool in Assagao                | Goa        |        0
 Modern 2BHK in HSR Layout                                | Bengaluru  |        0
 Newly Painted 1BHK in Maninagar Near Railway Station     | Ahmedabad  |        0
 OWNER                                                    | Bengaluru  |        0
 Pet-Friendly 1BHK near Indiranagar 100ft Road            | Bengaluru  |        0
 Pet-Friendly 2BHK near Whitefield                        | Bengaluru  |        0
 Quiet 1BHK near Manipal Hospital                         | Bengaluru  |        0
 Quiet 2BHK near Aundh with Study Nook                    | Pune       |        0
 Row House with Terrace in Baner                          | Pune       |        0
 Single Room with Attached Bath near Kukatpally Metro     | Hyderabad  |        0
 Skyline 3BHK in Whitefield with Gym and Pool             | Bengaluru  |        0
 Spacious 2BHK in Dwarka Sector 12 near Metro             | Delhi      |        0
 Student Studio near Symbiosis, Viman Nagar               | Pune       |        0
 Studio near Infopark with Kitchenette                    | Kochi      |        0
(59 rows)

===== A3. LEFT JOIN + IS NULL (anti-join): properties that have no reviews =====
                  title
-------------------------------------------
 2BHK with Balcony near SG Highway
 Student Studio near Symbiosis, Viman Nagar
 3BHK Sea-View Apartment in Bandra West
 Duplex 3BHK in Jubilee Hills
 Luxury Villa with Private Pool in Assagao
 3BHK Garden Apartment in Salt Lake
 4BHK Independent House in Besant Nagar
 Designer 3BHK on Golf Course Extension Road
 2BHK near Noida Sector 62 Metro
 2BHK Sea-Facing Apartment in Panjim
 Skyline 3BHK in Whitefield with Gym and Pool
 Modern 2BHK in HSR Layout
 1BHK in Gurugram Sector 56 near Rapid Metro
```

```
 Heritage 3BHK in Alipore
 Serviced Apartment in BKC
 4BHK Independent Floor in Vasant Vihar
 4BHK Sea-Facing Penthouse in Worli
 4BHK Penthouse in Indiranagar
 Independent Floor in Sector 21 Chandigarh
 Golf Course Road 2BHK Near Metro
 OWNER
 Row House with Terrace in Baner
 Farmhouse Retreat near Lonavala
(23 rows)
```

Figure 5.8: INNER JOIN, LEFT JOIN, anti-join

```
postgres@propfind

===== B1. GROUP BY + COUNT / AVG / MIN / MAX: price statistics per city =====
    city    | properties | avg_rent | min_rent | max_rent
------------+------------+----------+----------+-----------
 Mumbai     |          6 | 98416.67 | 14500.00 | 250000.00
 Delhi      |          2 | 68750.00 | 17500.00 | 120000.00
 Lonavala   |          1 | 48000.00 | 48000.00 |  48000.00
 Hyderabad  |          7 | 47857.14 |  8000.00 | 180000.00
 Gurugram   |          3 | 46333.33 | 19000.00 |  85000.00
 Goa        |          3 | 45666.67 | 16000.00 |  95000.00
 Jaipur     |          2 | 43250.00 | 11500.00 |  75000.00
 Bengaluru  |         13 | 31346.15 |  7500.00 | 110000.00
 Chennai    |          4 | 28625.00 |  4500.00 |  60000.00
 Kolkata    |          3 | 27000.00 | 14000.00 |  38000.00
 Pune       |          7 | 25642.86 |  6000.00 |  45000.00
 Chandigarh |          2 | 23000.00 | 13000.00 |  33000.00
 Kochi      |          3 | 17833.33 | 10500.00 |  31000.00
 Noida      |          1 | 16500.00 | 16500.00 |  16500.00
 Ahmedabad  |          2 | 15750.00 |  9500.00 |  22000.00
(15 rows)

===== B2. HAVING: cities with at least 2 listings and average rent above 20,000 =====
    city    | properties | avg_rent
------------+------------+----------
 Mumbai     |          6 | 98416.67
 Delhi      |          2 | 68750.00
 Hyderabad  |          7 | 47857.14
 Gurugram   |          3 | 46333.33
 Goa        |          3 | 45666.67
 Jaipur     |          2 | 43250.00
 Bengaluru  |         13 | 31346.15
 Chennai    |          4 | 28625.00
 Kolkata    |          3 | 27000.00
 Pune       |          7 | 25642.86
 Chandigarh |          2 | 23000.00
(11 rows)

===== B3. GROUP BY with SUM: revenue and count per booking status =====
  status   | bookings | total_value
-----------+----------+-------------
 PENDING   |        6 |   446500.00
 APPROVED  |        5 |   197500.00
 REJECTED  |        2 |   280000.00
 CANCELLED |        1 |   170000.00
(4 rows)
```

Figure 5.9: GROUP BY with COUNT/AVG/MIN/MAX/SUM and HAVING

```
-- C1. Scalar subquery: properties priced above the overall average
SELECT title, price_per_month
FROM properties
WHERE price_per_month > (SELECT AVG(price_per_month) FROM properties)
ORDER BY price_per_month DESC;
```

```
-- D1. Correlated: properties priced above the average of THEIR OWN city
SELECT p.title, p.city, p.price_per_month
FROM properties p
WHERE p.price_per_month > (
    SELECT AVG(p2.price_per_month)
    FROM properties p2
    WHERE p2.city = p.city
)
ORDER BY p.city, p.price_per_month DESC;
```

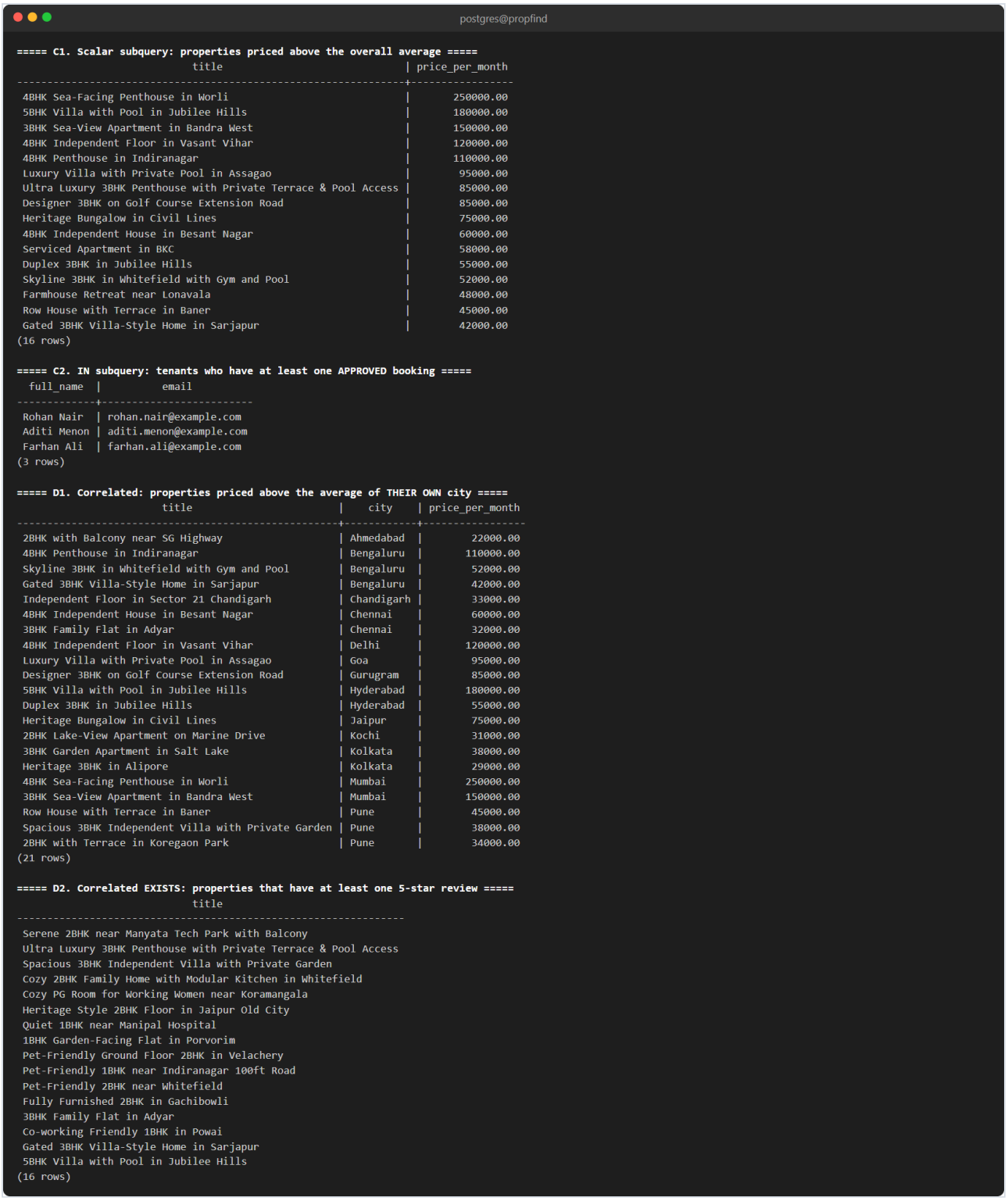


Figure 5.10: nested and correlated subqueries

```
-- F1. UPDATE with a subquery: raise rent 5% for owners with an APPROVED booking
BEGIN;
SELECT title, price_per_month AS before FROM properties WHERE city = 'Bengaluru' ORDER BY title;
UPDATE properties
SET price_per_month = ROUND(price_per_month * 1.05, 2)
WHERE id IN (SELECT property_id FROM bookings WHERE status = 'APPROVED');
SELECT title, price_per_month AS after, updated_at > created_at AS timestamp_trigger_fired
FROM properties WHERE id IN (SELECT property_id FROM bookings WHERE status = 'APPROVED');
ROLLBACK;
```

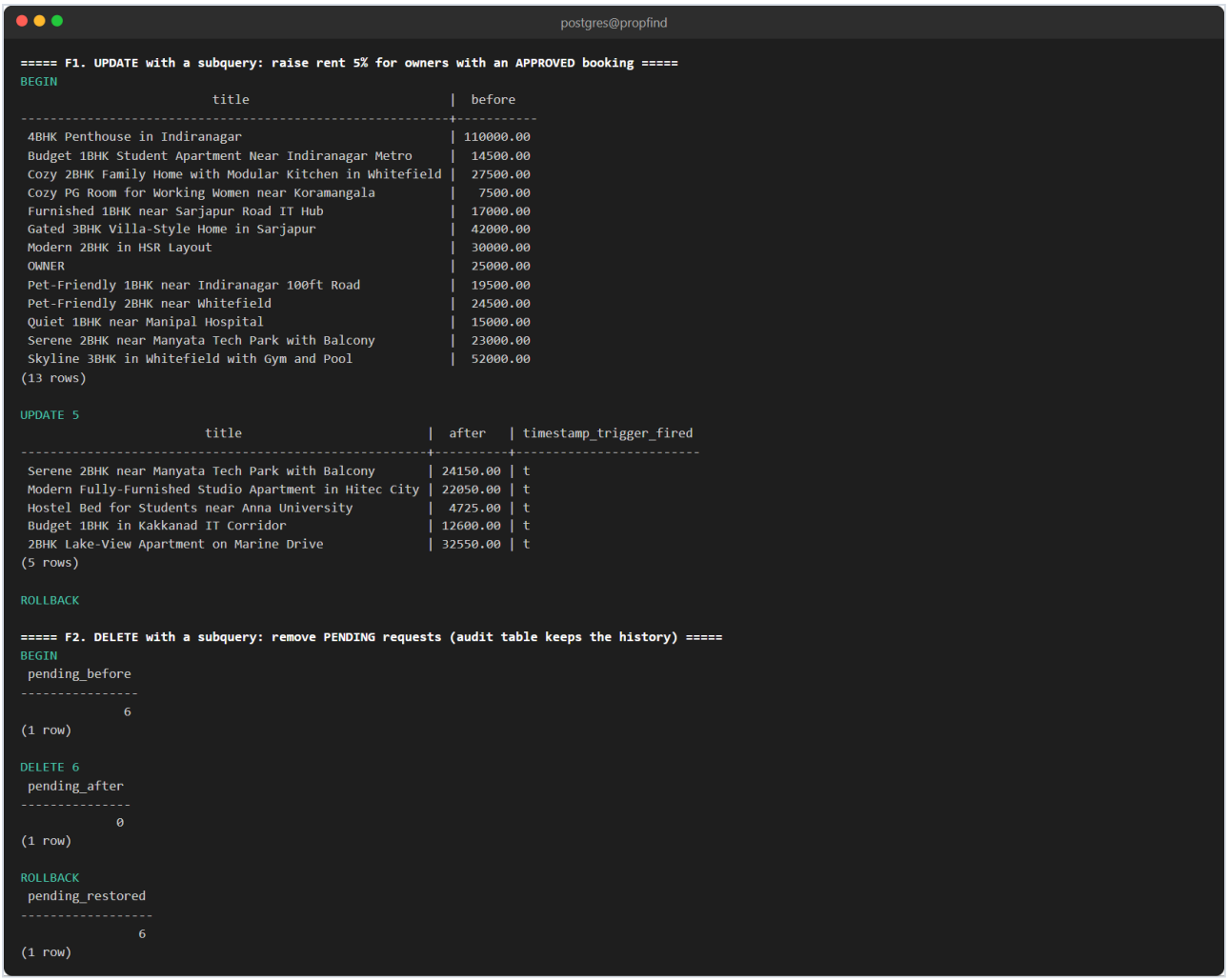


Figure 5.11: UPDATE and DELETE with subqueries, then ROLLBACK

5.5 Views, stored functions, triggers and indexes

```
-- G2. VIEW view_owner_analytics (GROUP BY + HAVING + SUM)
SELECT owner_name, total_properties, total_booking_requests, confirmed_bookings, total_confirmed_revenue
FROM view_owner_analytics
ORDER BY total_confirmed_revenue DESC;
```

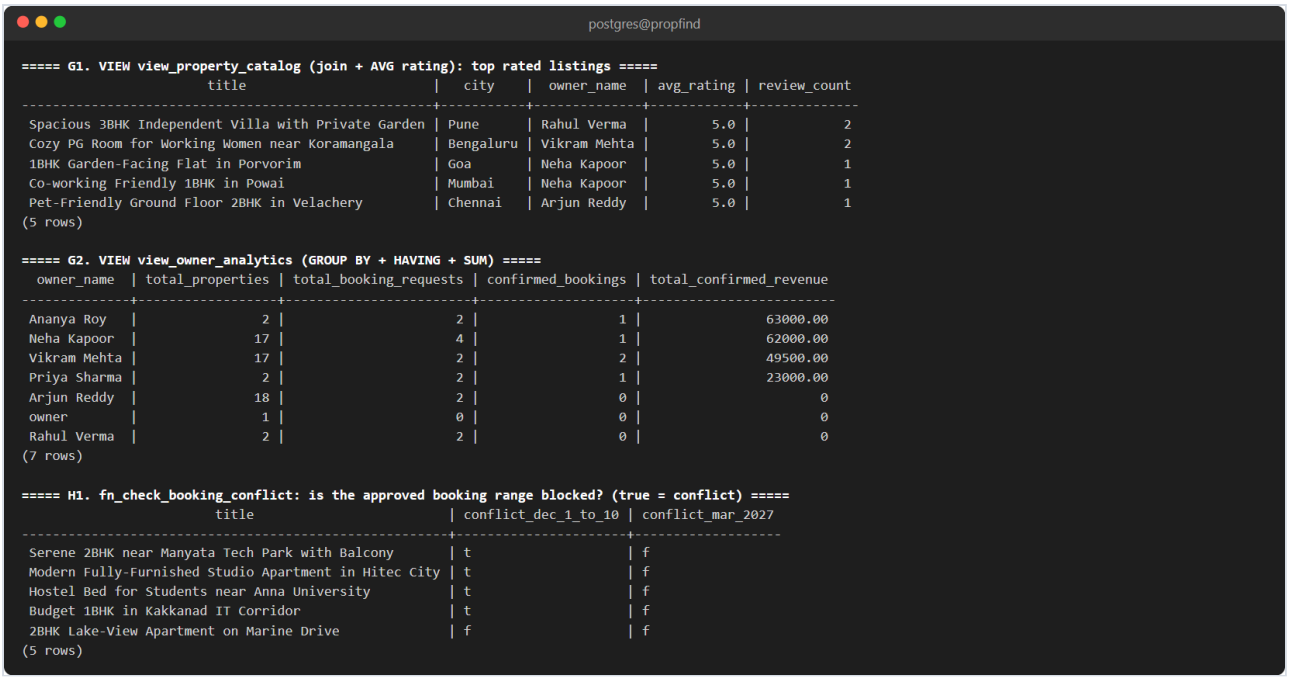


Figure 5.12: views (view_property_catalog, view_owner_analytics) and fn_check_booking_conflict



Figure 5.13: stored functions

db/schema.sql (double-booking trigger and booking state machine)

```sql
-- Trigger 3: Prevent overlapping confirmed bookings via trigger
CREATE OR REPLACE FUNCTION fn_trg_prevent_double_booking()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 'APPROVED' THEN
        IF fn_check_booking_conflict(NEW.property_id, NEW.start_date, NEW.end_date, NEW.id) THEN
            RAISE EXCEPTION 'Double booking conflict: Property % is already booked for date range % to %',
                NEW.property_id, NEW.start_date, NEW.end_date;
        END IF;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validate_booking_overlap
BEFORE INSERT OR UPDATE ON bookings
FOR EACH ROW
EXECUTE FUNCTION fn_trg_prevent_double_booking();

-- Trigger 4: Enforce the booking state machine at the database level
--   PENDING  -> APPROVED | REJECTED | CANCELLED
--   APPROVED -> CANCELLED
--   REJECTED / CANCELLED are terminal
CREATE OR REPLACE FUNCTION fn_trg_validate_booking_transition()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = OLD.status THEN
        RETURN NEW;
    END IF;
    IF (OLD.status = 'PENDING'  AND NEW.status IN ('APPROVED', 'REJECTED', 'CANCELLED'))
    OR (OLD.status = 'APPROVED' AND NEW.status = 'CANCELLED') THEN
        RETURN NEW;
    END IF;
    RAISE EXCEPTION 'Invalid booking status change: % -> %', OLD.status, NEW.status;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_validate_booking_transition ON bookings;
CREATE TRIGGER trg_validate_booking_transition
BEFORE UPDATE OF status ON bookings
FOR EACH ROW
EXECUTE FUNCTION fn_trg_validate_booking_transition();
```

Figure 5.14: double-booking trigger and booking state machine

db/schema.sql (payments table and payment trigger)

```sql
-- 9. Payments Table (Member 2): one PAID payment per approved booking
CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE RESTRICT,
    amount NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
    method payment_method NOT NULL,
    status payment_status NOT NULL DEFAULT 'PAID',
    transaction_ref VARCHAR(40) NOT NULL UNIQUE
        DEFAULT ('TXN-' || upper(substr(md5(random()::text || clock_timestamp()::text), 1, 12))),
    paid_at TIMESTAMPTZ,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT payments_paid_has_timestamp CHECK (status <> 'PAID' OR paid_at IS NOT NULL)
);
-- A booking can only be paid once (REFUNDED / FAILED rows do not block a new attempt)
CREATE UNIQUE INDEX uq_payments_one_paid_per_booking ON payments(booking_id) WHERE status = 'PAID';
CREATE INDEX idx_payments_booking ON payments(booking_id);

-- Trigger 5: Stamp paid_at and validate payments (BEFORE INSERT/UPDATE on payments)
--   * paid_at is set automatically when status becomes PAID
--   * a PAID payment is only allowed for an APPROVED booking and must equal total_amount
CREATE OR REPLACE FUNCTION fn_trg_payment_stamp_and_validate()
RETURNS TRIGGER AS $$
DECLARE
    b_status booking_status;
    b_total NUMERIC(10, 2);
BEGIN
    IF NEW.status = 'PAID' AND (TG_OP = 'INSERT' OR OLD.status IS DISTINCT FROM 'PAID') THEN
        SELECT status, total_amount INTO b_status, b_total FROM bookings WHERE id = NEW.booking_id;
        IF b_status IS DISTINCT FROM 'APPROVED' THEN
            RAISE EXCEPTION 'Payment rejected: booking % is %, only APPROVED bookings can be paid',
                NEW.booking_id, COALESCE(b_status::text, 'missing');
        END IF;
        IF NEW.amount <> b_total THEN
            RAISE EXCEPTION 'Payment rejected: amount % does not match booking total %', NEW.amount, b_total;
        END IF;
        NEW.paid_at := CURRENT_TIMESTAMP;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Figure 5.15: payments table and payment trigger

postgres@propfind

```
propfind_db=# SELECT tgname AS trigger, tgrelid::regclass AS on_table, ...
trg_properties_timestamp | properties | CREATE TRIGGER trg_properties_timestamp BEFORE UPDATE ON public.properties FOR EACH ROW EXECUTE F...
trg_audit_booking_status | bookings | CREATE TRIGGER trg_audit_booking_status AFTER INSERT OR UPDATE OF status ON public.bookings FOR EAC...
trg_bookings_timestamp | bookings | CREATE TRIGGER trg_bookings_timestamp BEFORE UPDATE ON public.bookings FOR EACH ROW EXECUTE FUNCTION ...
trg_refund_on_cancel | bookings | CREATE TRIGGER trg_refund_on_cancel AFTER UPDATE OF status ON public.bookings FOR EACH ROW EXECUTE FUNC...
trg_validate_booking_overlap | bookings | CREATE TRIGGER trg_validate_booking_overlap BEFORE INSERT OR UPDATE ON public.bookings FOR EACH...
trg_validate_booking_transition | bookings | CREATE TRIGGER trg_validate_booking_transition BEFORE UPDATE OF status ON public.bookings FO...
trg_payment_stamp_and_validate | payments | CREATE TRIGGER trg_payment_stamp_and_validate BEFORE INSERT OR UPDATE ON public.payments FOR ...
```

Figure 5.16: all 7 triggers installed in the database

Indexes: B-tree on `properties(owner_id)`, `(city)`, `(price_per_month)`, `(status)`, `bookings(property_id, start_date, end_date)`, `bookings(tenant_id)`, `payments(booking_id)`, `booking_audit(booking_id, changed_at)`; a **partial unique index** on `payments(booking_id) WHERE status = 'PAID'`; and the pgvector **HNSW** index described in section 6.

5.6 Transactions and integrity tests

The booking flow runs as one transaction: lock the property row, check conflicts, compute the total, insert. The test script demonstrates BEGIN/COMMIT, ROLLBACK, SAVEPOINT, atomicity (a failing statement undoes the whole transaction), row locking, and 22 negative tests that each must be rejected by the database.

lib/services/booking-service.ts (transactional booking with row lock)

```ts
export async function createBookingRequest(
  data: BookingRequestInput
): Promise<{ success: boolean; bookingId?: string; message: string }> {
  const start = parseDate('Start date', data.start_date);
  const end = parseDate('End date', data.end_date);
  if (end <= start) throw new Error('End date must be after the start date');

  const today = new Date();
  today.setUTCHours(0, 0, 0, 0);
  if (start < today) throw new Error('Start date cannot be in the past');

  return await withTransaction(async (client) => {
    await client.query(`SELECT set_config('app.user_id', $1, true)`, [data.tenant_id]);

    // Step 1: lock the property row so concurrent requests are serialised
    const propCheck = await client.query(
      `SELECT id, price_per_month, status, owner_id
       FROM properties
       WHERE id = $1
       FOR UPDATE`,
      [data.property_id]
    );

    if (propCheck.rows.length === 0) throw new Error('Property not found');

    const property = propCheck.rows[0];
    if (property.owner_id === data.tenant_id) {
      throw new Error('You cannot book your own property');
    }
    if (property.status !== 'AVAILABLE') {
      throw new Error(`Property is currently ${property.status} and cannot be booked`);
    }

    // Step 2: date conflict check against already APPROVED bookings (stored function)
    const conflictCheck = await client.query(
      `SELECT fn_check_booking_conflict($1, $2::date, $3::date) AS has_conflict`,
      [data.property_id, data.start_date, data.end_date]
    );
    if (conflictCheck.rows[0]?.has_conflict) {
      throw new Error('Property has an overlapping confirmed booking for these dates');
    }

    // Step 3: a tenant may not hold two live requests for overlapping dates on one property
    const dup = await client.query(
      `SELECT 1 FROM bookings
       WHERE property_id = $1 AND tenant_id = $2
         AND status IN ('PENDING', 'APPROVED')
         AND (start_date, end_date) OVERLAPS ($3::date, $4::date)
       LIMIT 1`,
      [data.property_id, data.tenant_id, data.start_date, data.end_date]
    );
    if (dup.rows.length > 0) {
      throw new Error('You already have an active request for these dates on this property');
    }

    // Step 4: total = price_per_month * months (30-day blocks, minimum 1)
    const days = Math.round((end.getTime() - start.getTime()) / 86_400_000);
    const months = Math.max(1, Math.round(days / 30));
    const totalAmount = Number(property.price_per_month) * months;

    // Step 5: insert as PENDING
    const insertRes = await client.query(
```

Figure 5.17: transactional booking creation with SELECT ... FOR UPDATE (lib/services/booking-service.ts)

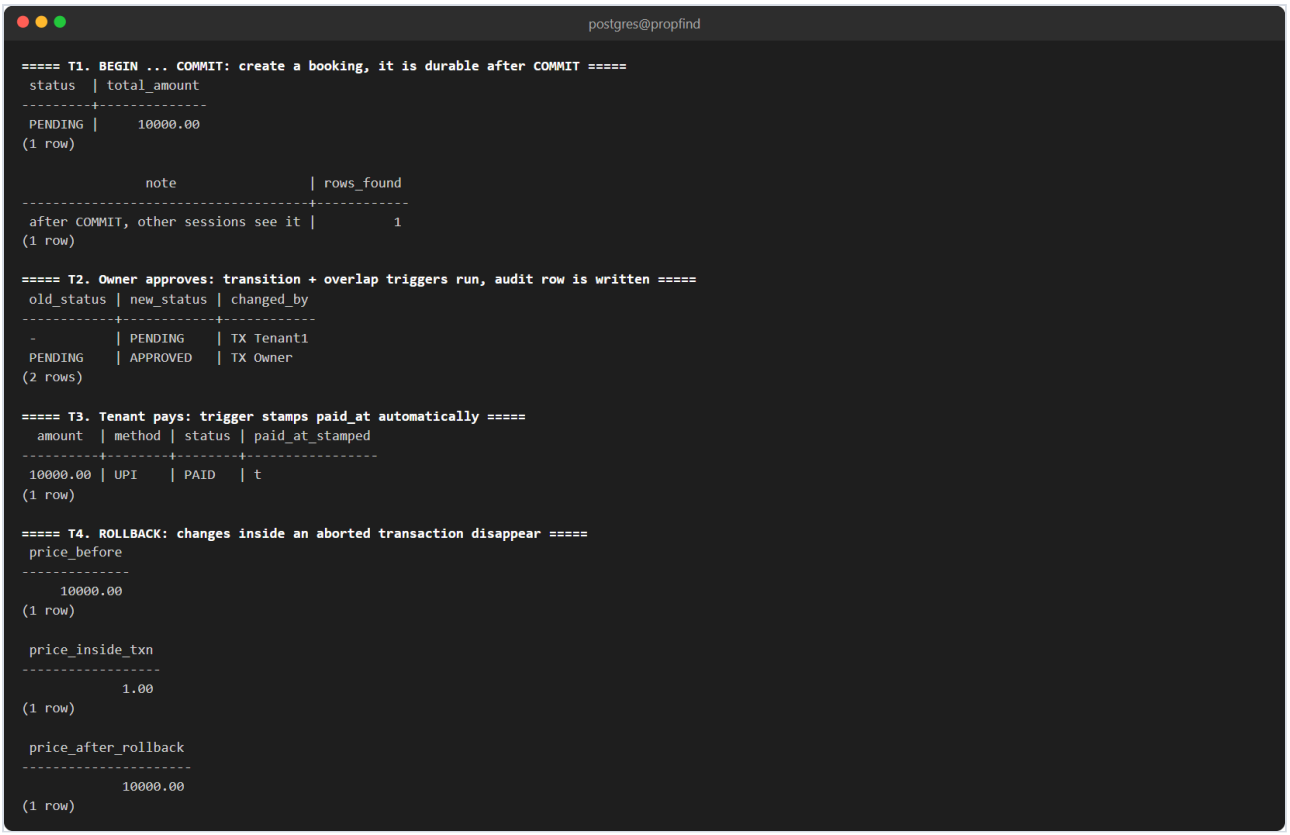


Figure 5.18: COMMIT, trigger-written audit row, payment timestamp, ROLLBACK

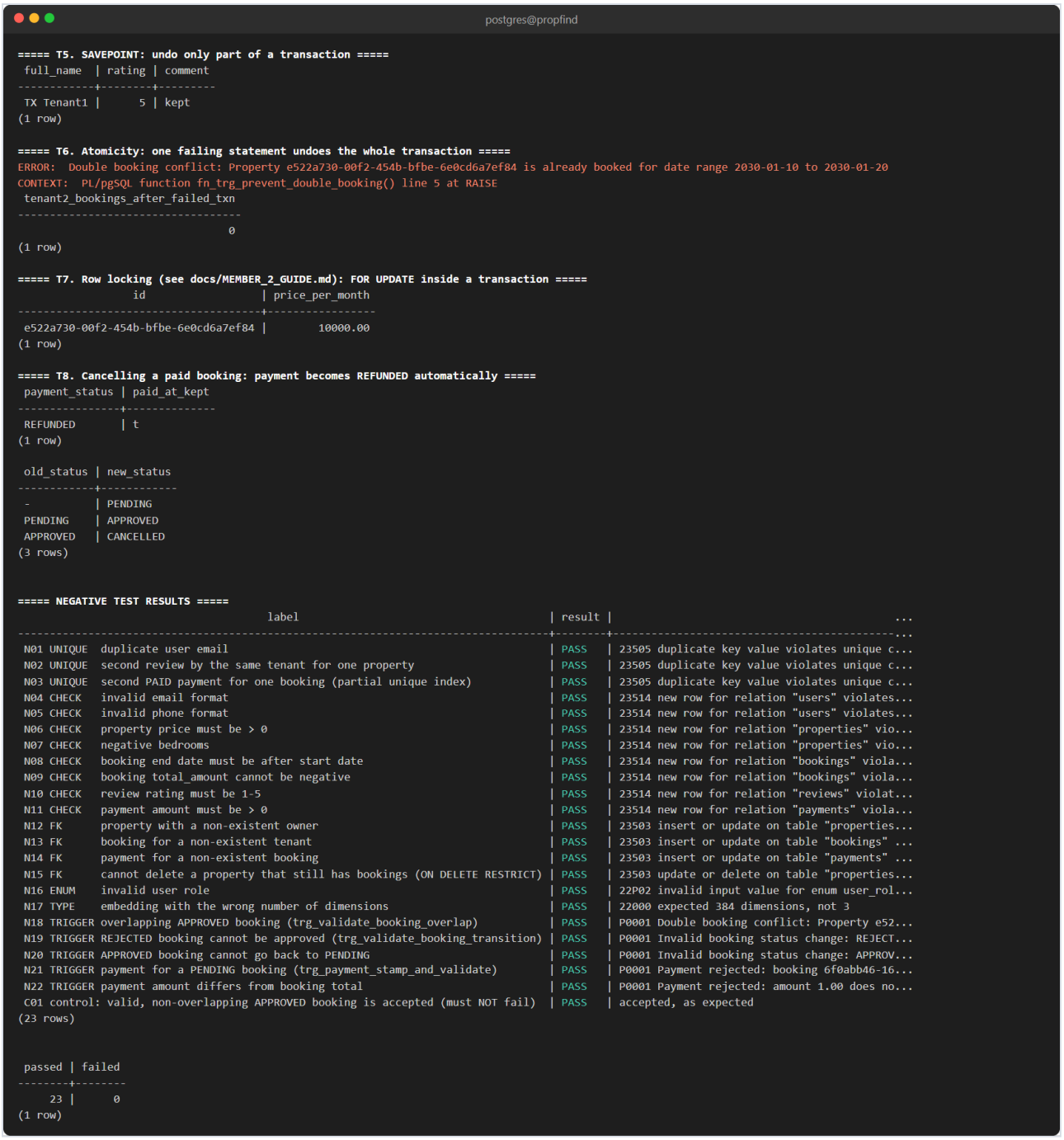


Figure 5.19: SAVEPOINT, atomicity and automatic refund

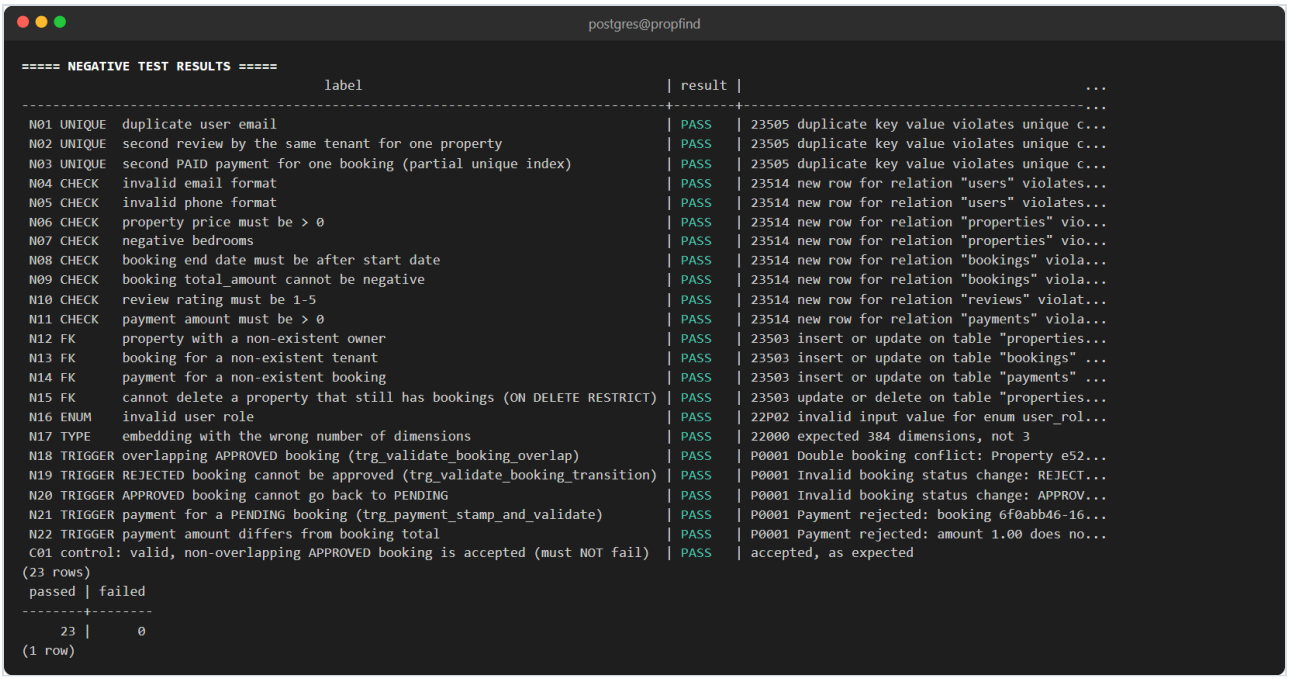


Figure 5.20: 22 negative tests (UNIQUE, CHECK, FOREIGN KEY, ENUM, vector dimension, triggers) and a positive control: 23 / 23 PASS

6. Vector Database Commands (NoSQL paradigm)

Choice and justification. The requirements allow SQL plus either NoSQL or a vector database. PropFind uses **PostgreSQL with pgvector**: listings are structured (price, city, bedrooms) *and* text-heavy (descriptions, amenities), so one engine can store both, filter exactly with SQL, and rank by meaning with vectors. This keeps embeddings consistent with the relational rows (same transaction, foreign key, cascade delete) and avoids running a second database. A document store such as MongoDB would add a second system without helping the semantic requirement.

6.1 What is embedded, with which model, and how it is stored

Question	Answer
What data is converted to embeddings?	For each listing: title, amenities and description joined into one text (amenities placed right after the title so short facts like "Pet Friendly" are not diluted). Each search query is embedded too.
Which model?	all-MiniLM-L6-v2 (sentence-transformers), run locally through @huggingface/transformers with mean pooling and L2 normalisation. 384 dimensions. Optional OpenAI path if a key is set.
How are vectors stored?	Table property_embeddings: embedding vector(384), plus model_name and the exact indexed_content text. One row per property, FK with ON DELETE CASCADE.
Similarity metric	Cosine distance (operator <=>); similarity = 1 - distance, shown as a percentage in the UI.
How are results retrieved?	Query text -> embedding -> stored function fn_search_properties_semantic orders by cosine distance using the HNSW index, after relational filters (city, max price, bedrooms, status).
Metadata and hybrid filters	Budget, bedrooms and city written in the sentence are parsed (lib/query-parser.ts) into exact SQL conditions; only the descriptive remainder is embedded.

6.2 Commands

```
-- enable the extension
CREATE EXTENSION IF NOT EXISTS vector;

-- vector table (384 = all-MiniLM-L6-v2)
CREATE TABLE property_embeddings (
    property_id UUID PRIMARY KEY REFERENCES properties(id) ON DELETE CASCADE,
    embedding vector(384) NOT NULL,
    model_name VARCHAR(100) NOT NULL DEFAULT 'all-MiniLM-L6-v2',
    indexed_content TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- HNSW index for approximate nearest-neighbour search on cosine distance
CREATE INDEX idx_property_embeddings_hnsw
ON property_embeddings USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- insert (the application passes the vector as a bind parameter)
```

```
INSERT INTO property_embeddings (property_id, embedding, model_name, indexed_content)
VALUES ($1, $2::vector, 'all-MiniLM-L6-v2', $3);
```

```
CREATE OR REPLACE FUNCTION fn_search_properties_semantic(
    p_query_vector vector(384),
    p_match_threshold DOUBLE PRECISION DEFAULT 0.20,
    p_match_count INT DEFAULT 10,
    p_city VARCHAR DEFAULT NULL,
    p_max_price NUMERIC DEFAULT NULL,
    p_bedrooms INT DEFAULT NULL
)
RETURNS TABLE (
    id UUID,
    title VARCHAR,
    description TEXT,
    property_type VARCHAR,
    price_per_month NUMERIC,
    bedrooms INT,
    bathrooms INT,
    area_sqft INT,
    address TEXT,
    city VARCHAR,
    status property_status,
    amenities TEXT[],
    image_url TEXT,
    owner_name VARCHAR,
    avg_rating NUMERIC,
    similarity_score DOUBLE PRECISION
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        vpc.id,
        vpc.title,
        vpc.description,
        vpc.property_type,
        vpc.price_per_month,
        vpc.bedrooms,
        vpc.bathrooms,
        vpc.area_sqft,
        vpc.address,
        vpc.city,
        vpc.status,
        vpc.amenities,
        vpc.image_url,
        vpc.owner_name,
        vpc.avg_rating,
        -- Cosine distance (<=>) ranges from 0 (identical) to 2 (opposite)
        -- Similarity score is normalized to [0, 1]
        (1 - (pe.embedding <=> p_query_vector)) AS similarity_score
    FROM property_embeddings pe
    JOIN view_property_catalog vpc ON pe.property_id = vpc.id
    WHERE (1 - (pe.embedding <=> p_query_vector)) >= p_match_threshold
      AND (p_city IS NULL OR LOWER(vpc.city) = LOWER(p_city))
      AND (p_max_price IS NULL OR vpc.price_per_month <= p_max_price)
      AND (p_bedrooms IS NULL OR vpc.bedrooms = p_bedrooms)
      AND vpc.status = 'AVAILABLE'
    ORDER BY pe.embedding <=> p_query_vector ASC
    LIMIT p_match_count;
END;
$$ LANGUAGE plpgsql;

-- ------------------------------------------------------------------------
```

```
-- I1. Listings most similar to the Manyata Tech Park apartment (nearest neighbours)
SELECT p.title,
       ROUND((1 - (e.embedding <=> ref.embedding))::numeric, 3) AS cosine_similarity
FROM property_embeddings e
JOIN properties p ON p.id = e.property_id
CROSS JOIN (
    SELECT pe.embedding
    FROM property_embeddings pe JOIN properties pp ON pp.id = pe.property_id
    WHERE pp.title LIKE 'Serene 2BHK%'
) AS ref
WHERE p.title NOT LIKE 'Serene 2BHK%'
ORDER BY e.embedding <=> ref.embedding
LIMIT 3;
```

```
-- I3. Hybrid: vector ranking + relational filters (city and price)
SELECT p.title, p.city, p.price_per_month,
       ROUND((1 - (e.embedding <=> ref.embedding))::numeric, 3) AS similarity
FROM property_embeddings e
JOIN properties p ON p.id = e.property_id
CROSS JOIN (
    SELECT pe.embedding FROM property_embeddings pe JOIN properties pp ON pp.id = pe.property_id
    WHERE pp.title LIKE 'Modern Fully-Furnished Studio%'
) AS ref
WHERE p.city = 'Bengaluru' AND p.price_per_month <= 30000
ORDER BY e.embedding <=> ref.embedding
LIMIT 3;
```

postgres@propfind

```
propfind_db=# \d property_embeddings
                                Table "public.property_embeddings"
     Column      |           Type           | Collation | Nullable |                Default
-----------------+--------------------------+-----------+----------+---------------------------------------
 property_id     | uuid                     |           | not null |
 embedding       | vector(384)              |           | not null |
 model_name      | character varying(100)   |           | not null | 'all-MiniLM-L6-v2'::character varying
 indexed_content | text                     |           | not null |
 created_at      | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
 updated_at      | timestamp with time zone |           | not null | CURRENT_TIMESTAMP
Indexes:
    "property_embeddings_pkey" PRIMARY KEY, btree (property_id)
    "idx_property_embeddings_hnsw" hnsw (embedding vector_cosine_ops) WITH (m='16', ef_construction='64')
Foreign-key constraints:
    "property_embeddings_property_id_fkey" FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE CASCADE
```

Figure 6.1: \d property_embeddings: vector(384) column and HNSW index

postgres@propfind

```
===== I1. Listings most similar to the Manyata Tech Park apartment (nearest neighbours) =====
                      title                       | cosine_similarity
--------------------------------------------------+-------------------
 2BHK near Noida Sector 62 Metro                  |             0.574
 2BHK with Balcony near SG Highway                |             0.556
 Spacious 3BHK Independent Villa with Private Garden |             0.541
(3 rows)

===== I2. Stored function fn_search_properties_semantic with a stored vector as the query =====
                      title                       |   city    | similarity
--------------------------------------------------+-----------+------------
 Budget 1BHK Student Apartment Near Indiranagar Metro | Bengaluru |      1.000
 4BHK Penthouse in Indiranagar                    | Bengaluru |      0.710
 Pet-Friendly 1BHK near Indiranagar 100ft Road    | Bengaluru |      0.701
(3 rows)

===== I3. Hybrid: vector ranking + relational filters (city and price) =====
                       title                        |   city    | price_per_month | similarity
----------------------------------------------------+-----------+-----------------+------------
 Modern 2BHK in HSR Layout                          | Bengaluru |        30000.00 |      0.434
 Cozy 2BHK Family Home with Modular Kitchen in Whitefield | Bengaluru |        27500.00 |      0.424
 Furnished 1BHK near Sarjapur Road IT Hub           | Bengaluru |        17000.00 |      0.411
(3 rows)

===== I4. Vector index in use: the HNSW index exists on property_embeddings =====
         indexname          |                                                                                                indexdef                                                                                                ...
----------------------------+--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------...
 idx_property_embeddings_hnsw | CREATE INDEX idx_property_embeddings_hnsw ON public.property_embeddings USING hnsw (embedding vector_cosi...
(1 row)
```

Figure 6.2: nearest neighbours (<=>), the search function, hybrid vector + relational filter, HNSW index

```
-- J1. Full-text search: listings mentioning quiet AND garden (ranked)
SELECT title,
       ROUND(ts_rank(to_tsvector('english', title || ' ' || description),
                     to_tsquery('english', 'quiet & garden'))::numeric, 4) AS rank
FROM properties
WHERE to_tsvector('english', title || ' ' || description) @@ to_tsquery('english', 'quiet & garden')
ORDER BY rank DESC;
```

```
postgres@propfind

===== J1. Full-text search: listings mentioning quiet AND garden (ranked) =====
                     title                      |  rank
------------------------------------------------+--------
 Spacious 3BHK Independent Villa with Private Garden | 0.1036
 3BHK Garden Apartment in Salt Lake              | 0.1012
 1BHK Garden-Facing Flat in Porvorim             | 0.0000
(3 rows)

===== K1. LEFT JOIN payments: approved bookings and whether they are paid =====
                  id                  |                        title                        | total_amount | payment
--------------------------------------+-----------------------------------------------------+--------------+---------
 891c727e-66fc-44d4-ab70-b9a733670aca | Serene 2BHK near Manyata Tech Park with Balcony     |     23000.00 | UNPAID
 078834bb-e544-43b8-ab7e-1e5f38e173e1 | Modern Fully-Furnished Studio Apartment in Hitec City |     63000.00 | PAID
 4d3bfb47-251a-4f86-b8b3-f0a64dabb736 | Hostel Bed for Students near Anna University        |     13500.00 | PAID
 26c7f614-b5ee-4aca-a58b-25909c2cd77c | Budget 1BHK in Kakkanad IT Corridor                 |     36000.00 | UNPAID
 72d725c5-c92f-4cf9-99e4-44a3f70a91dd | 2BHK Lake-View Apartment on Marine Drive            |     62000.00 | PAID
(5 rows)
```

Figure 6.3: full-text search (tsvector) and a payments / audit query

6.3 Application code for semantic search

lib/embeddings.ts (real all-MiniLM-L6-v2 embedding, mean pooling + normalisation)

```ts
/** all-MiniLM-L6-v2 embedding or semantic vectorizer fallback */
export async function generateLocalEmbedding(text: string): Promise<number[]> {
  try {
    const extractor = await getExtractor();
    if (extractor) {
      const out = await extractor(text, { pooling: 'mean', normalize: true });
      const vec = Array.from(out.data as Float32Array);
      if (vec.length === EMBEDDING_DIMENSION) {
        return vec;
      }
    }
  } catch (err) {
    useOfflineFallback = true;
  }
  return generateSemanticHashEmbedding(text);
}
```

Figure 6.4: generating a 384-dimension embedding (lib/embeddings.ts)

lib/query-parser.ts (budget, bedrooms and city extracted from the sentence)

```ts
function toRupees(raw: string, unit?: string): number {
  const n = parseFloat(raw.replace(/,/g, ''));
  const u = (unit || '').toLowerCase();
  if (u === 'k' || u === 'thousand') return Math.round(n * 1_000);
  if (u === 'l' || u === 'lac' || u === 'lacs' || u === 'lakh' || u === 'lakhs') return Math.round(n * 100_000);
  return Math.round(n);
}

const fmt = (n: number) => '₹' + n.toLocaleString('en-IN');

export function parseSearchQuery(input: string, knownCities: string[] = []): ParsedQuery {
  let text = ` ${input} `;
  const applied: string[] = [];
  const result: ParsedQuery = { text: '', isGeneric: false, applied };

  // Budget ceiling
  let m = text.match(MAX_RE);
  if (m) {
    const v = toRupees(m[1], m[2]);
    // a bare small number ("under 3") is more likely bedrooms than rupees: ignore it
    if (v >= 1000) {
      result.maxPrice = v;
      applied.push(`under ${fmt(v)}`);
      text = text.replace(MAX_RE, ' ');
    }
  }

  // Budget floor
  m = text.match(MIN_RE);
  if (m) {
    const v = toRupees(m[1], m[2]);
    if (v >= 1000) {
      result.minPrice = v;
      applied.push(`above ${fmt(v)}`);
      text = text.replace(MIN_RE, ' ');
    }
  }

  // Loose phrasing ("20k budget", "rent 15000") sets a ceiling if none was found yet
  if (result.maxPrice === undefined && result.minPrice === undefined) {
    m = text.match(LOOSE_RE);
    if (m) {
      const raw = m[1] ?? m[3];
      const unit = m[1] ? m[2] : m[4];
      const v = toRupees(raw, unit);
      if (v >= 1000) {
        result.maxPrice = v;
        applied.push(`under ${fmt(v)}`);
        text = text.replace(LOOSE_RE, ' ');
      }
    }
  }
```

Figure 6.5: extracting budget, bedrooms and city from the sentence (lib/query-parser.ts)

lib/services/property-service.ts (hybrid search: parsed filters + vector ranking)

```ts
  // Semantic path: embed only the descriptive part of the sentence
  const queryEmbedding = await getEmbedding(parsed.text);
  const vectorStr = toVectorLiteral(queryEmbedding);

  const sql = `
    SELECT
      id, title, description, property_type, price_per_month, bedrooms, bathrooms,
      area_sqft, address, city, status, amenities, image_url, owner_name, avg_rating,
      ROUND((similarity_score * 100)::numeric, 1)::float AS similarity_score
    FROM fn_search_properties_semantic($1::text::vector, 0.20, $2, $3, $4, $5)
    WHERE ($6::numeric IS NULL OR price_per_month >= $6)
  `;
  const result = await query<Property>(sql, [
    vectorStr,
    limit ?? 30,
    city ?? null,
    maxPrice ?? null,
    bedrooms ?? null,
    minPrice ?? null,
  ]);
  return { properties: result.rows.map(mapRow), applied, semanticText: parsed.text };
}
```

Figure 6.6: hybrid search service (lib/services/property-service.ts)

NoSQL note. No separate NoSQL database is used. The only non-relational structure is the PostgreSQL array column `properties.amenities TEXT[]` and the vector column; both live inside the same transactional engine.

7. Next.js – Project Tree Structure

```
propfind/
├── app/   # Next.js App Router: pages and API route handlers
│   ├── api/   # REST API (route handlers)
│   │   ├── auth/   # register, login, logout, me
│   │   │   ├── login/route.ts
│   │   │   ├── logout/route.ts
│   │   │   ├── me/route.ts
│   │   │   └── register/route.ts
│   │   ├── bookings/   # create / list bookings, status change, pay, history
│   │   │   ├── [id]/
│   │   │   │   ├── pay/route.ts
│   │   │   │   └── route.ts
│   │   │   └── route.ts
│   │   ├── owner/properties/route.ts   # owner-only data (own properties)
│   │   ├── properties/route.ts   # catalog + create listing (with embedding)
│   │   └── search/route.ts   # semantic + hybrid search
│   ├── dashboard/   # owner and tenant dashboards
│   │   ├── owner/   # owner portal + detail pages (properties, requests, confirmed, revenue)
│   │   │   ├── confirmed/page.tsx
│   │   │   ├── properties/page.tsx
│   │   │   ├── requests/page.tsx
│   │   │   ├── revenue/page.tsx
│   │   │   └── page.tsx
│   │   └── tenant/page.tsx
│   ├── login/page.tsx   # login page
│   ├── properties/new/page.tsx   # new-listing form
│   ├── register/page.tsx   # registration page
│   ├── globals.css
│   ├── layout.tsx
│   └── page.tsx
├── components/   # shared React components
│   ├── Footer.tsx
│   ├── Navbar.tsx
│   ├── OwnerSubpageHeader.tsx
│   ├── PropertyCard.tsx
│   └── PropertyModal.tsx
├── db/   # SQL: schema, demo queries, transaction tests
│   ├── queries.sql
│   ├── schema.sql
│   └── transactions_tests.sql
├── docs/   # project documentation and this report
│   ├── report/
│   │   ├── images/ (screenshots)
│   │   ├── PropFind_Project_Report.docx
│   │   └── PropFind_Project_Report.pdf
│   ├── ARCHITECTURE.md
│   ├── DATABASE_DESIGN.md
│   ├── DEV_LOGS.md
│   ├── HOSTED_DATABASE.md
│   ├── MEMBER_2_GUIDE.md
│   └── TEAM_DIVISION.md
├── lib/   # server-side libraries
│   ├── services/   # booking and property services (SQL access)
│   │   ├── booking-service.ts
│   │   └── property-service.ts
│   ├── auth.ts
│   ├── db.ts
│   ├── embeddings.ts
│   ├── jwt.ts
│   └── query-parser.ts
├── scripts/   # db:init, db:seed, seed data, parser tests
│   ├── init-db.mjs
```

```
│   ├── seed-data.mjs
│   ├── seed.mjs
│   └── test-parser.mjs
├── .env.example   # configuration template (no secrets)
├── .gitignore
├── DBMS Level 3 Mini_Project Requirements draft.docx
├── docker-compose.yml   # PostgreSQL 16 + pgvector container
├── middleware.ts   # role-based route protection
├── next.config.mjs
├── package.json
├── postcss.config.mjs
├── README.md
├── tailwind.config.ts
└── tsconfig.json
```

8. Sample Screenshots: Application Code and Demo

8.1 Application code

middleware.ts (role-based route protection)

```ts
/**
 * Role-based access control: runs before every protected page loads.
 * (API routes additionally re-check the role themselves via requireRole().)
 */
import { NextResponse, type NextRequest } from 'next/server';
import { verifyToken, COOKIE, type Role } from '@/lib/jwt';

const rules: { prefix: string; roles: Role[] }[] = [
  { prefix: '/dashboard/owner', roles: ['OWNER', 'ADMIN'] },
  { prefix: '/dashboard/tenant', roles: ['TENANT', 'ADMIN'] },
  { prefix: '/properties/new', roles: ['OWNER', 'ADMIN'] },
];

export async function middleware(req: NextRequest) {
  const rule = rules.find((r) => req.nextUrl.pathname.startsWith(r.prefix));
  if (!rule) return NextResponse.next();

  const session = await verifyToken(req.cookies.get(COOKIE)?.value);
  if (!session) {
    const url = new URL('/login', req.url);
    url.searchParams.set('next', req.nextUrl.pathname);
    return NextResponse.redirect(url);
  }
  if (!rule.roles.includes(session.role)) {
    return NextResponse.redirect(new URL('/', req.url));
  }
  return NextResponse.next();
}

export const config = {
  matcher: ['/dashboard/:path*', '/properties/new'],
};
```

Figure 8.1: role-based route protection (middleware.ts)

lib/jwt.ts + lib/auth.ts (JWT session with jose, httpOnly cookie)

```ts
export async function createToken(s: Session): Promise<string> {
  return new SignJWT({ name: s.name, role: s.role })
    .setProtectedHeader({ alg: 'HS256' })
    .setSubject(s.userId)
    .setIssuedAt()
    .setExpirationTime('7d')
    .sign(secret());
}

export async function requireRole(...roles: Role[]): Promise<Session | null> {
  const s = await getSession();
  if (!s || !roles.includes(s.role)) return null;
  return s;
}

export async function setSessionCookie(res: NextResponse, s: Session) {
  res.cookies.set(COOKIE, await createToken(s), {
    httpOnly: true,
    sameSite: 'lax',
    path: '/',
    maxAge: COOKIE_MAX_AGE,
    secure: process.env.COOKIE_SECURE === 'true', // enable when served over HTTPS
  });
}
```

Figure 8.2: JWT session helpers and role check (lib/jwt.ts, lib/auth.ts)

Further code screenshots: Figures 5.4, 5.14, 5.15, 5.17 (SQL and booking transaction) and 6.4 to 6.6 (embeddings, parser, search).

8.2 Demo: search

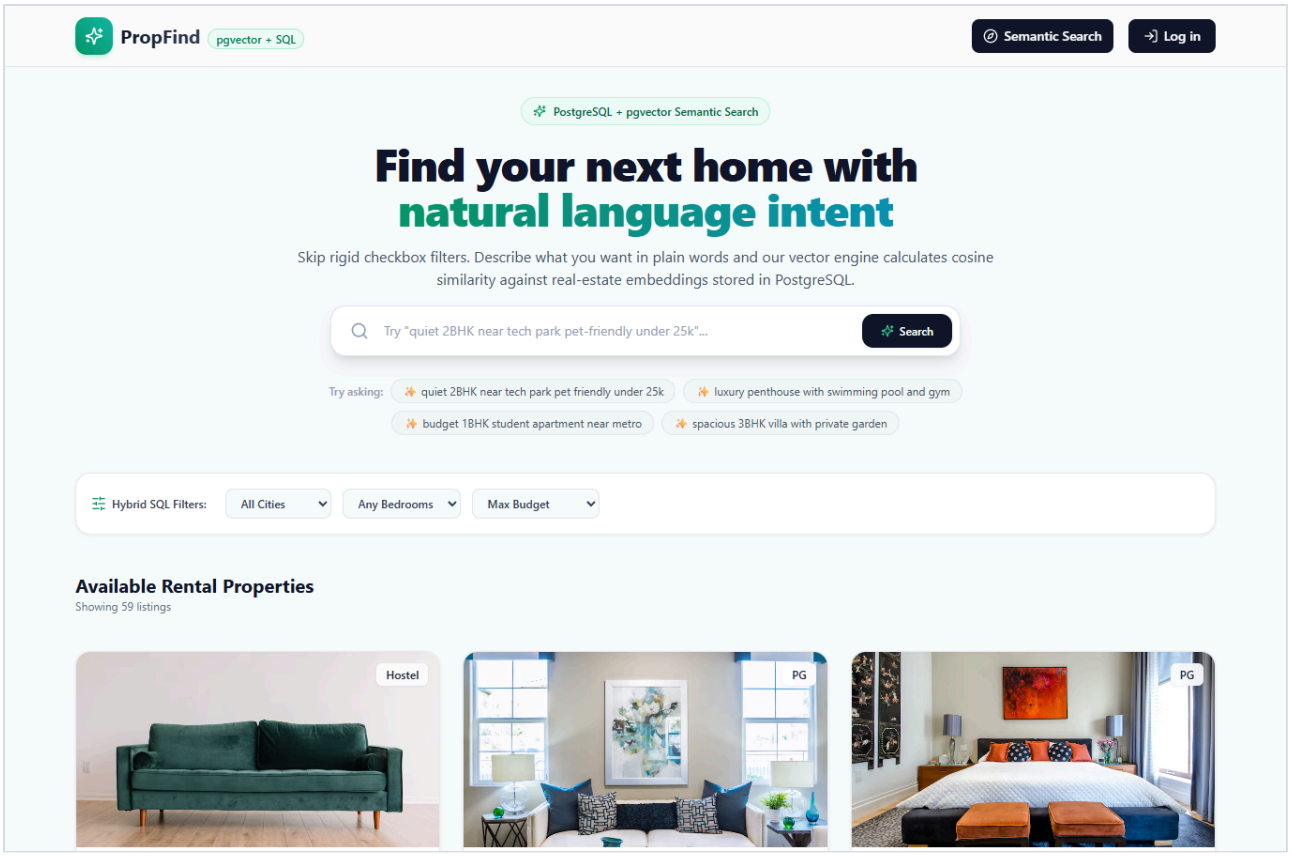


Figure 8.3: home page

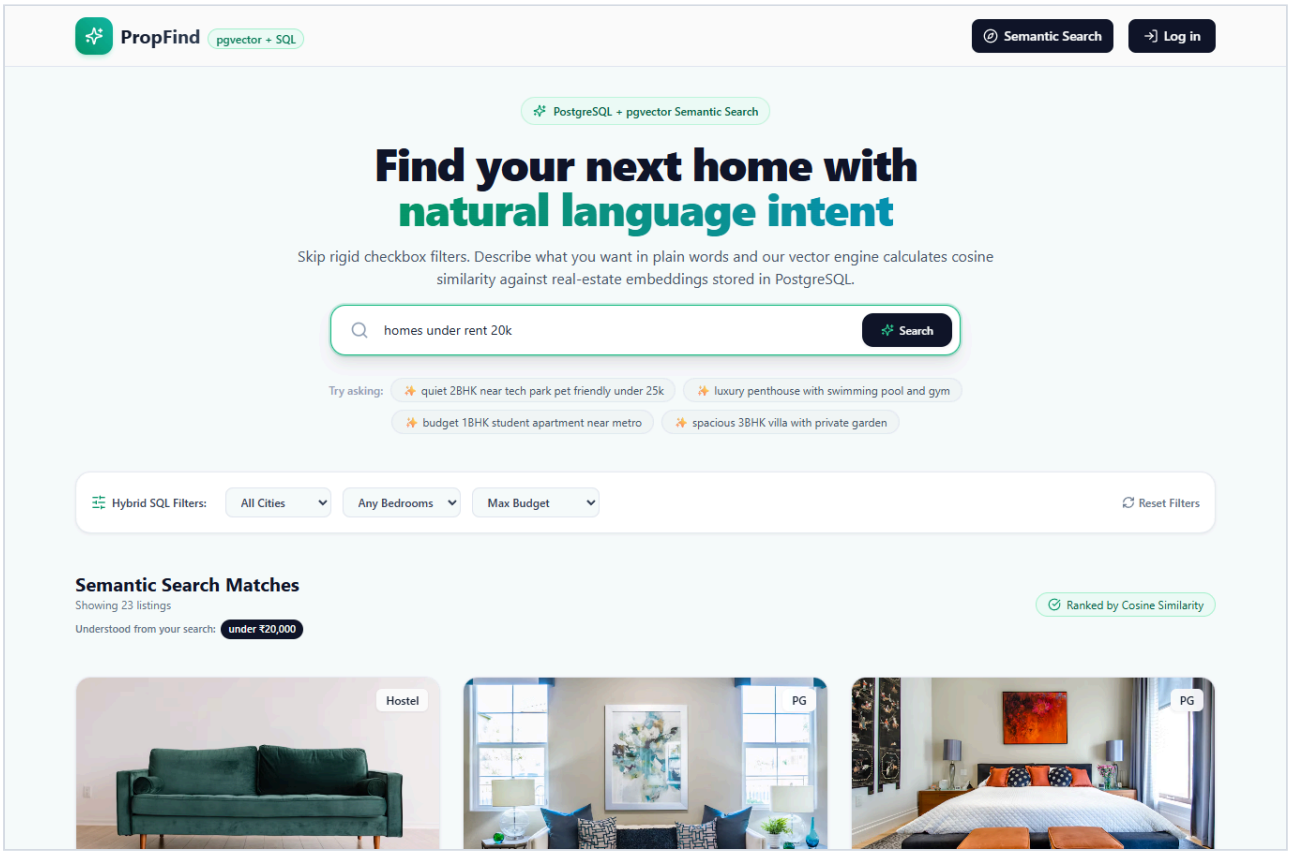


Figure 8.4: "homes under rent 20k": 23 results, all under ₹20,000, with the understood constraint shown

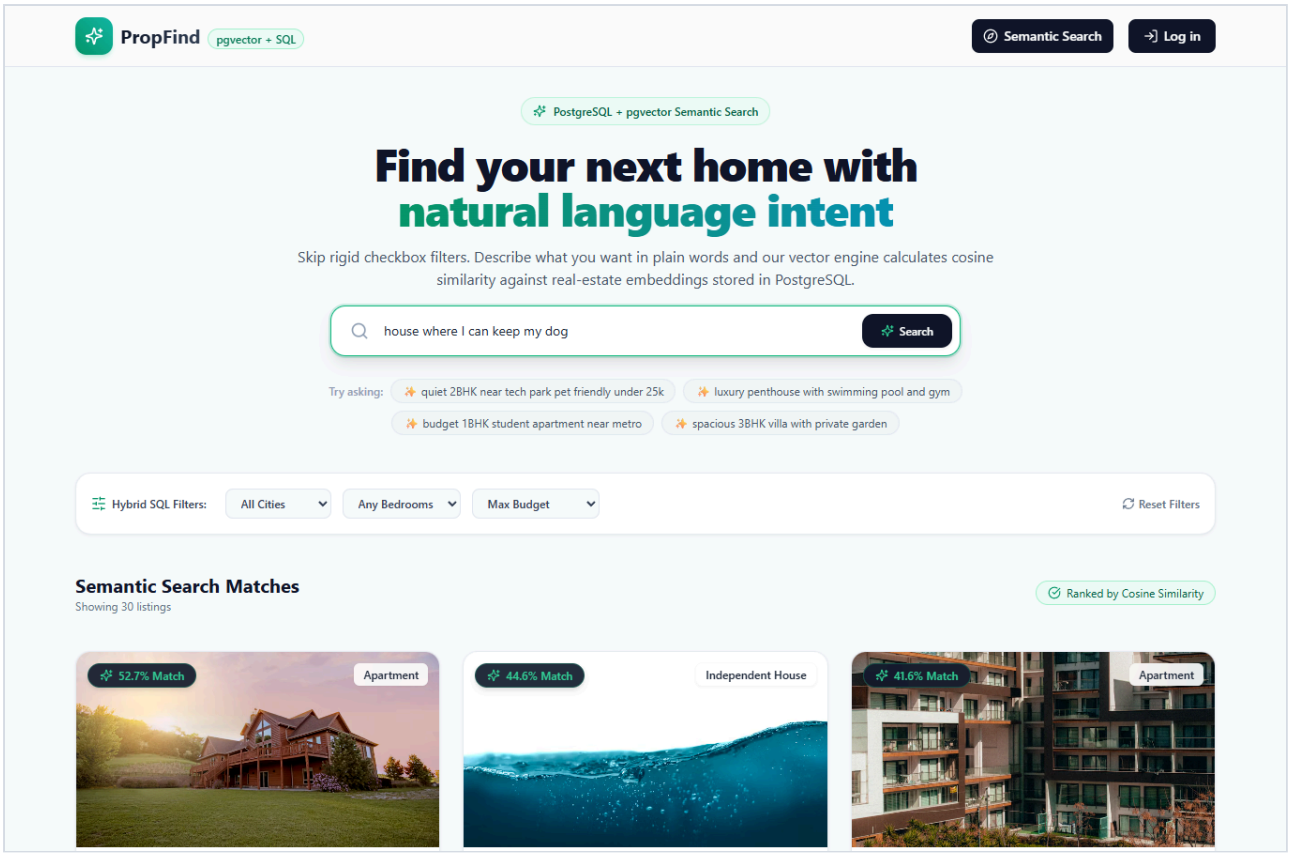


Figure 8.5: "house where I can keep my dog": semantic ranking puts pet-friendly homes first

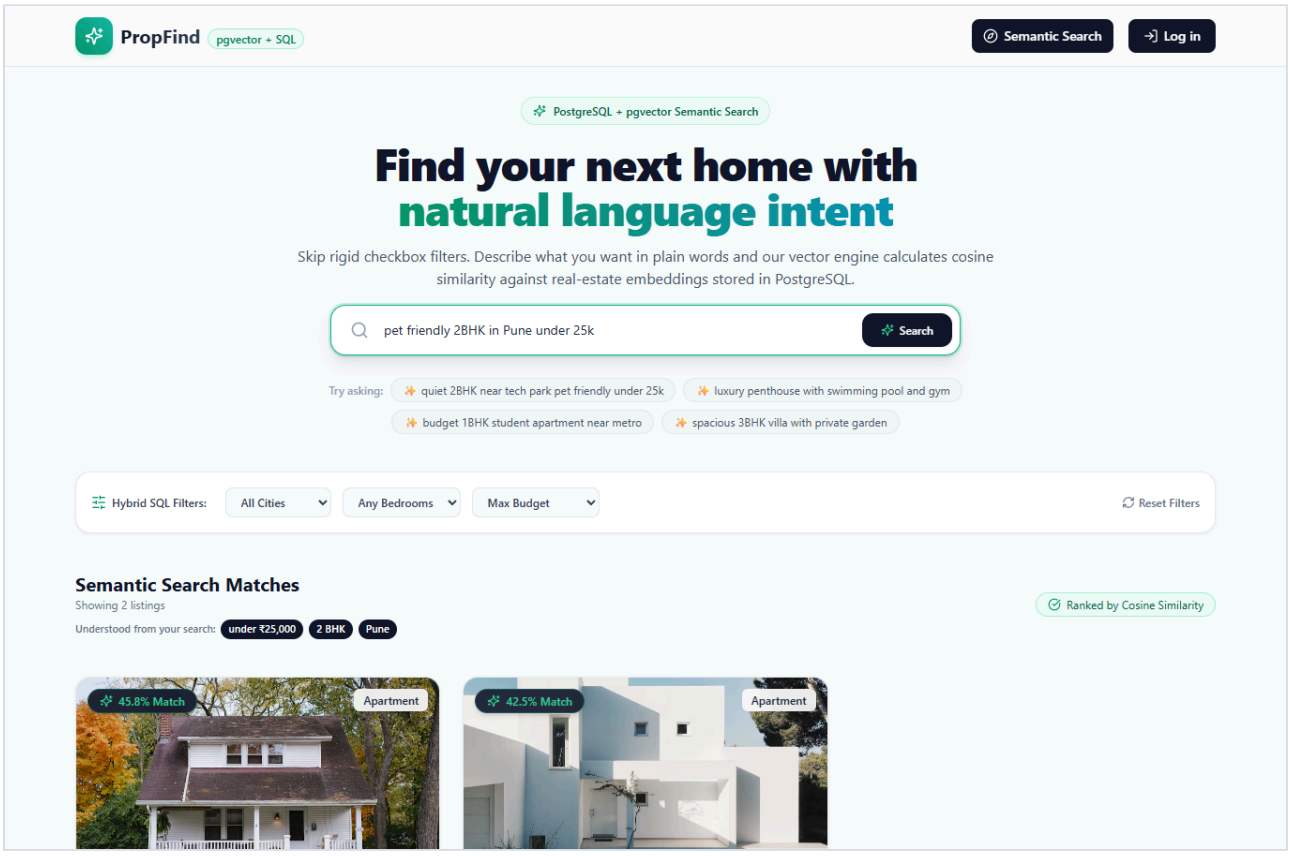


Figure 8.6: "pet friendly 2BHK in Pune under 25k": bedrooms, city and budget parsed into SQL filters

8.3 Demo: accounts and booking

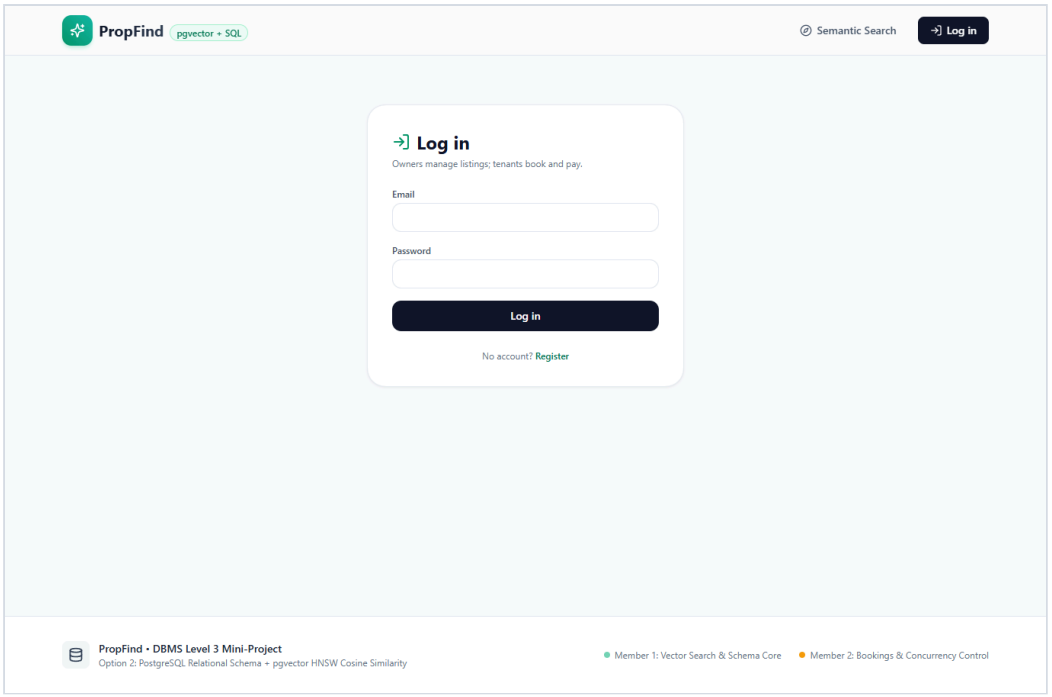


Figure 8.7: login

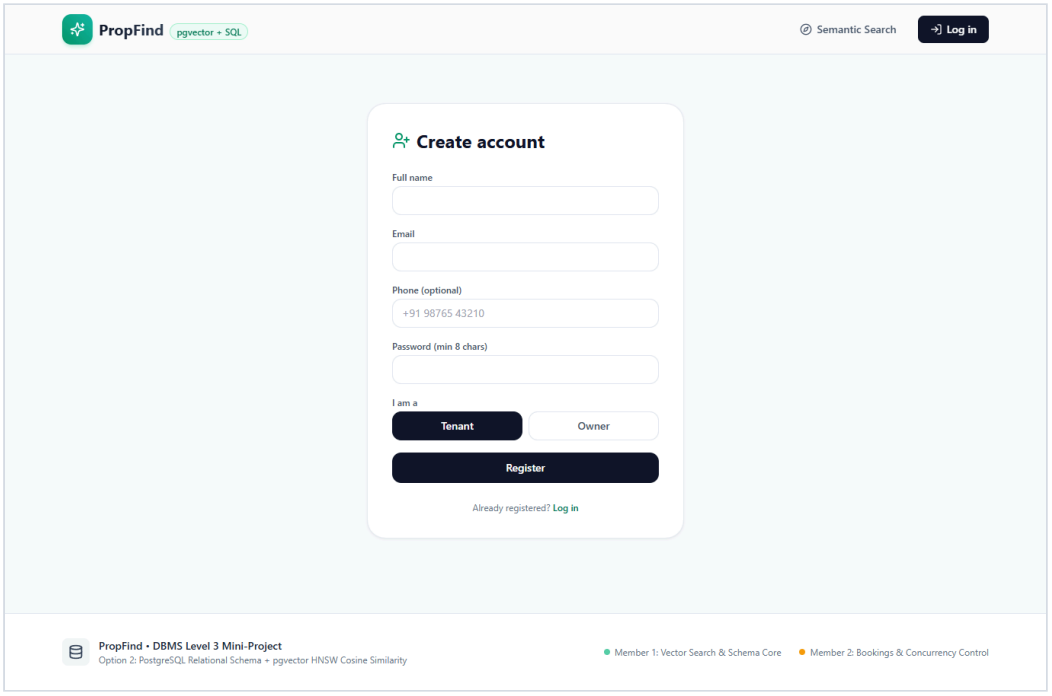


Figure 8.8: registration (owner or tenant)

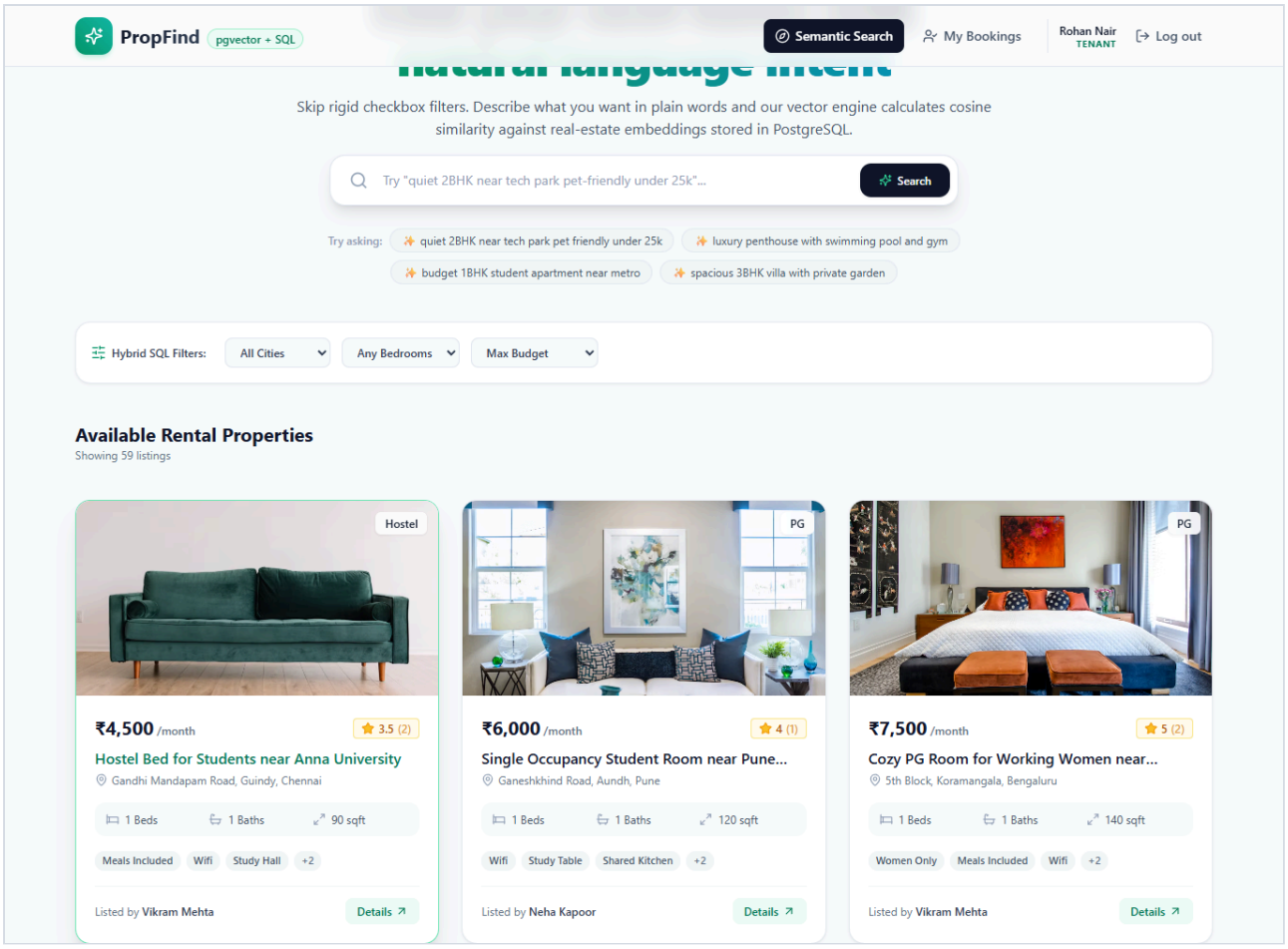


Figure 8.9: listing details

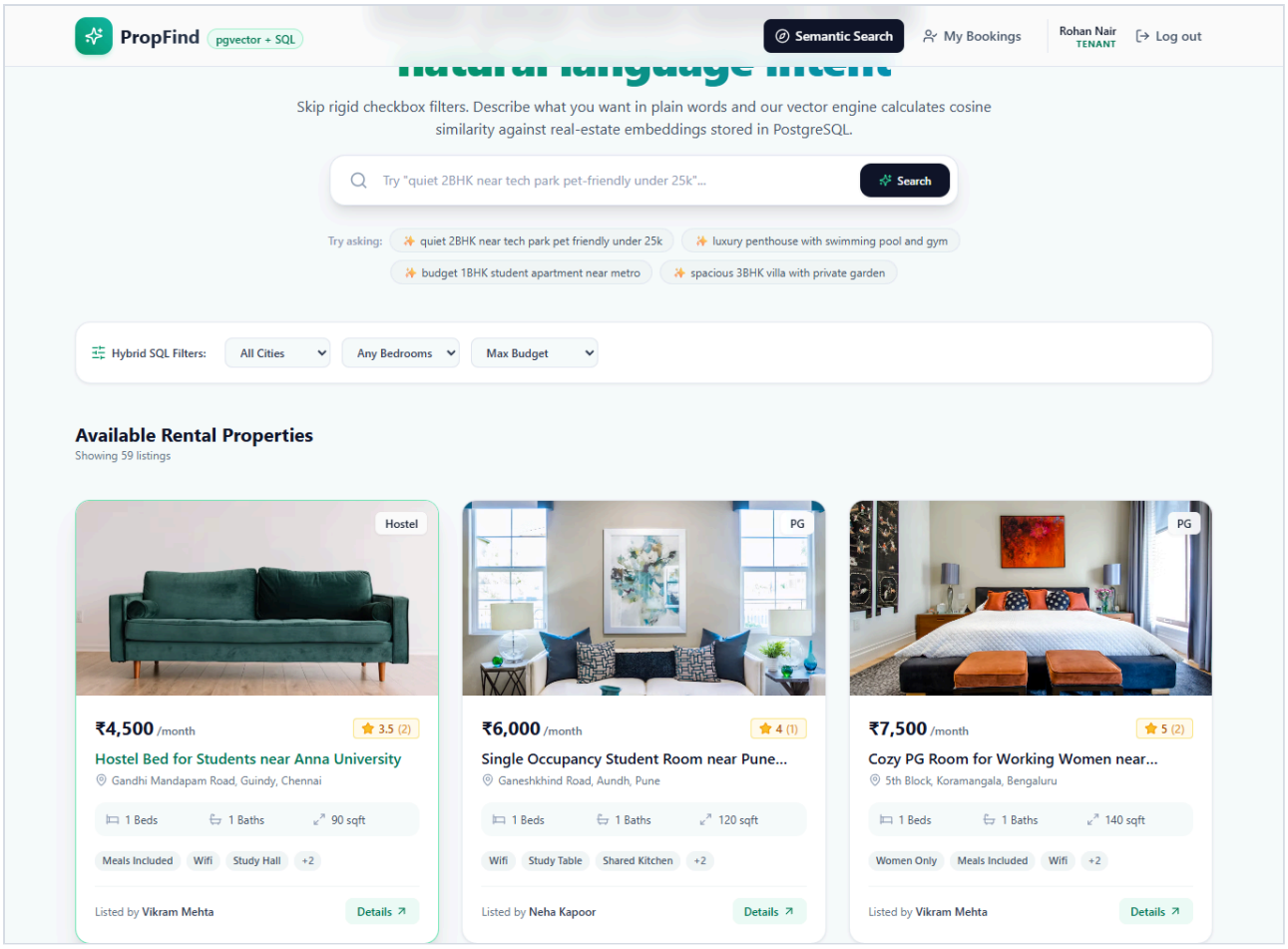


Figure 8.10: booking request form (runs as a PostgreSQL transaction)

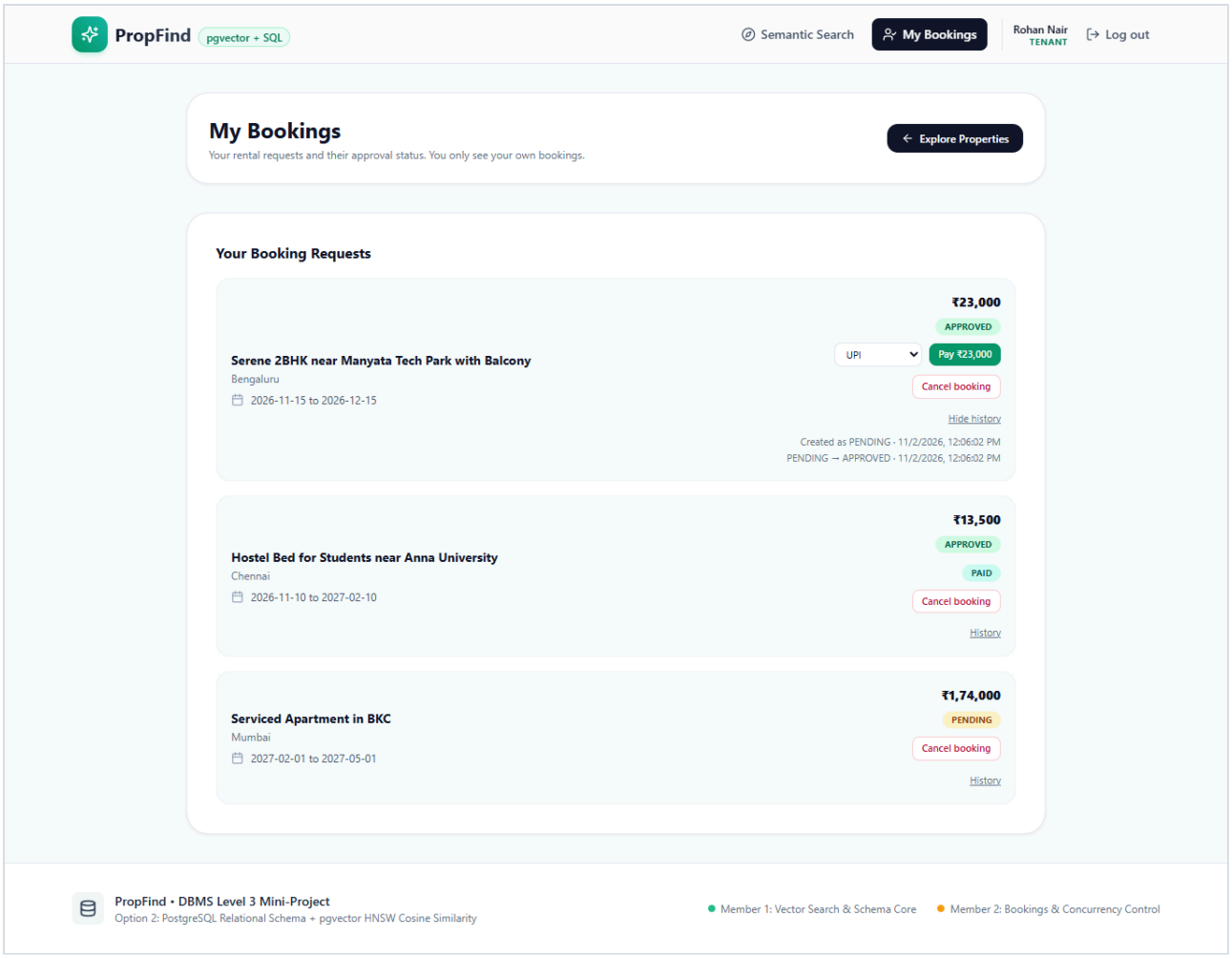


Figure 8.11: tenant dashboard: own bookings, Pay, Cancel and status history from the audit table

8.4 Demo: owner portal

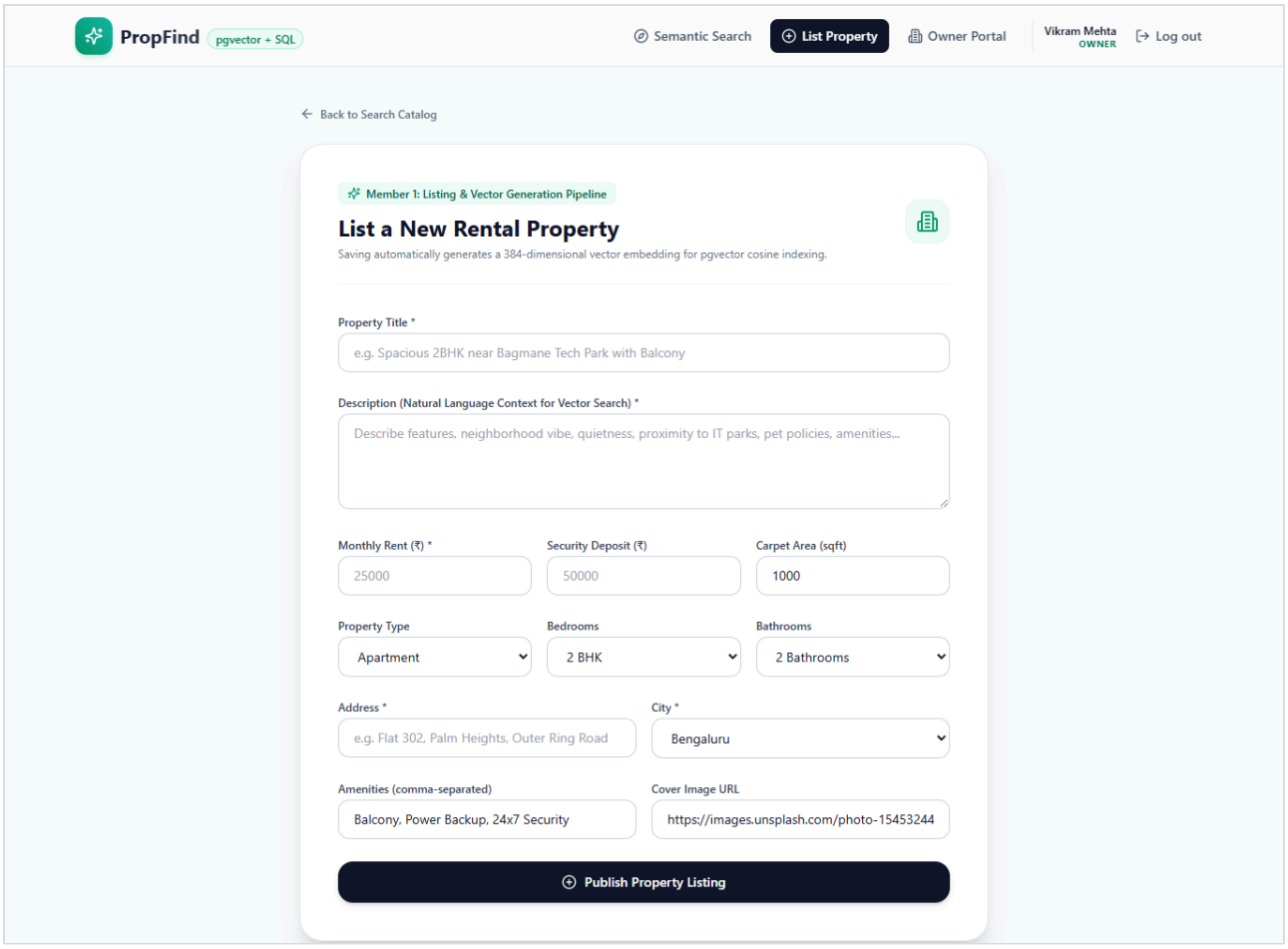


Figure 8.12: listing form (embedding generated in the same transaction)

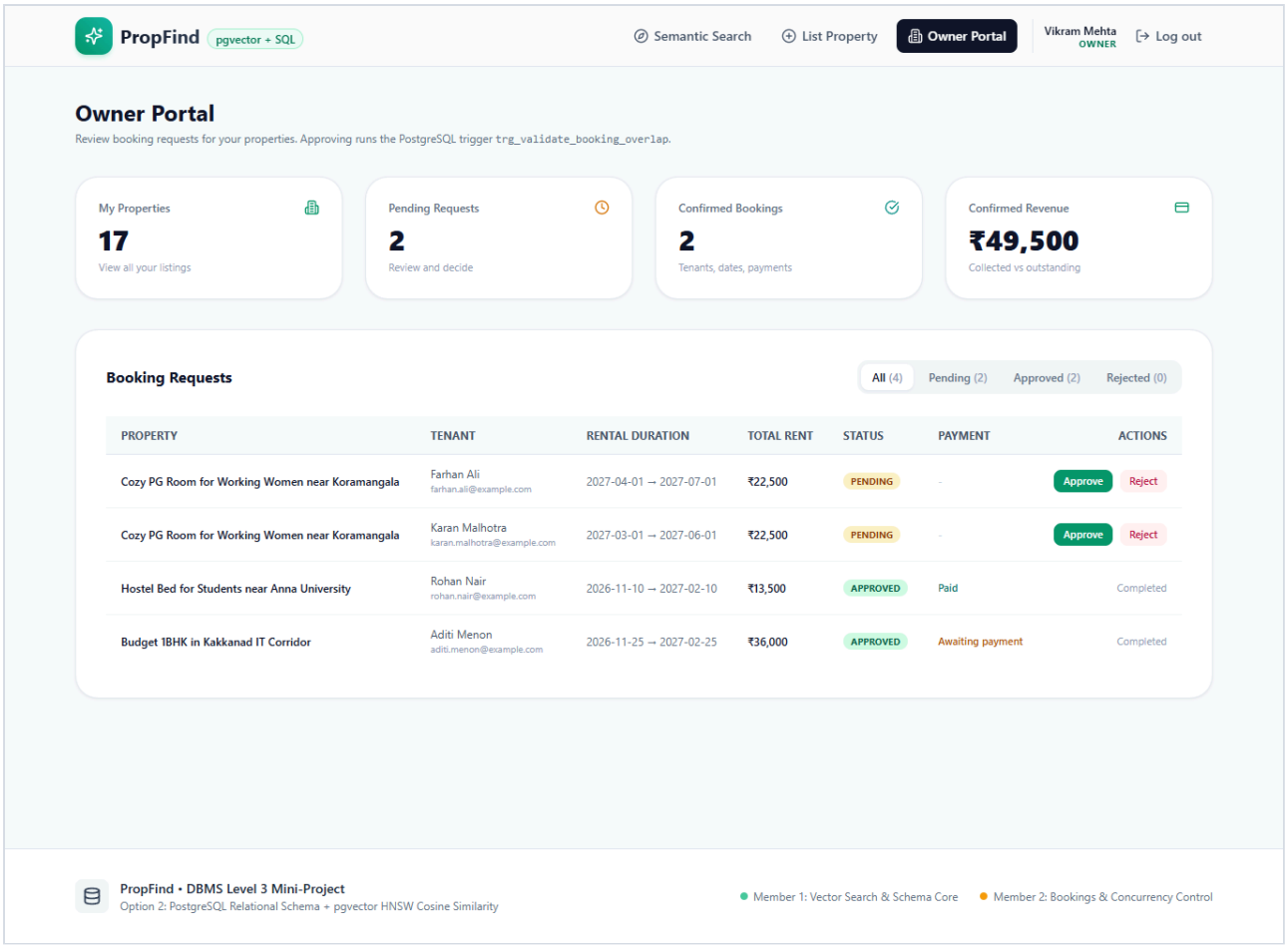


Figure 8.13: owner portal: four clickable cards (from view_owner_analytics) and booking tabs

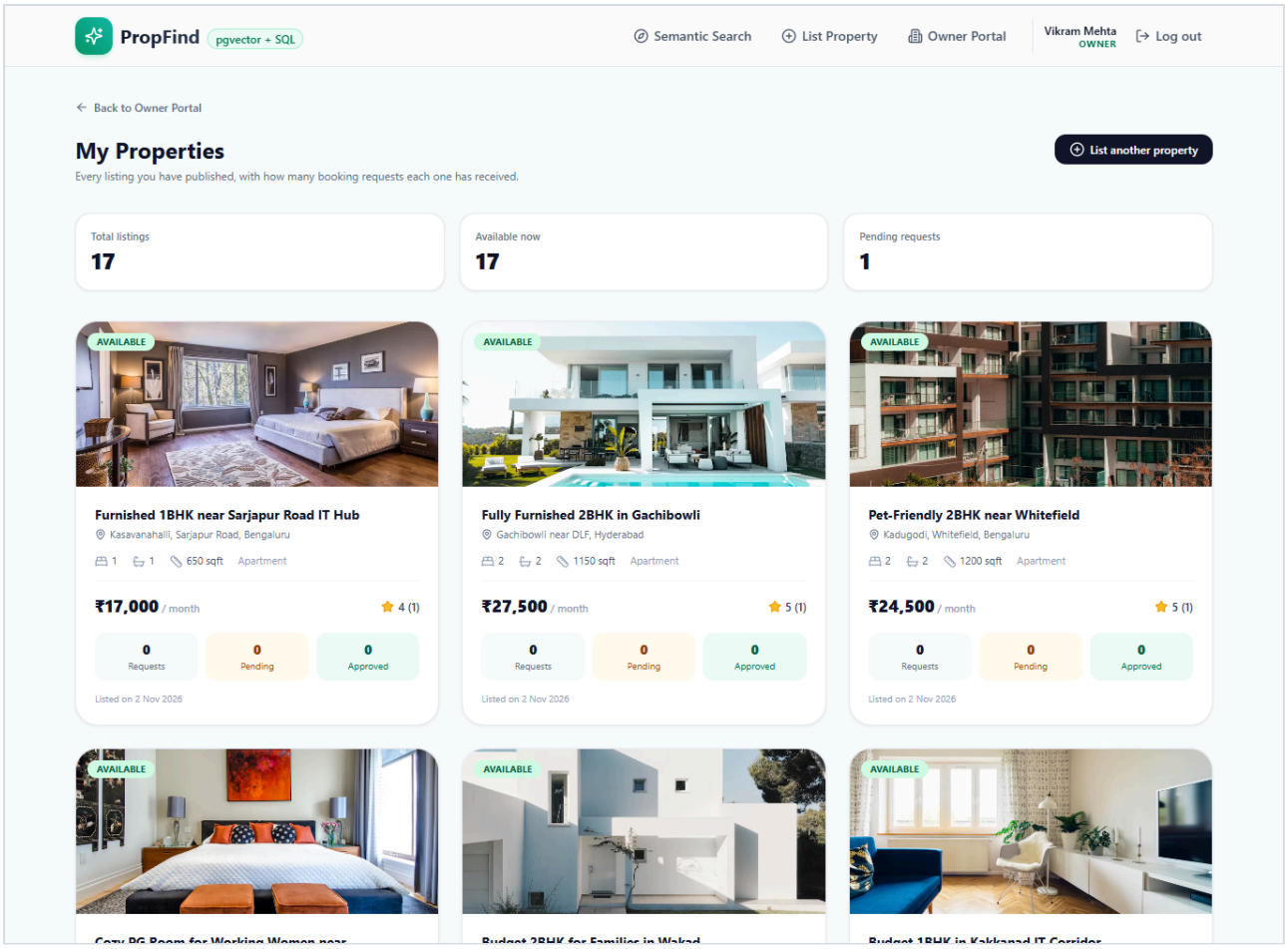


Figure 8.14: My Properties

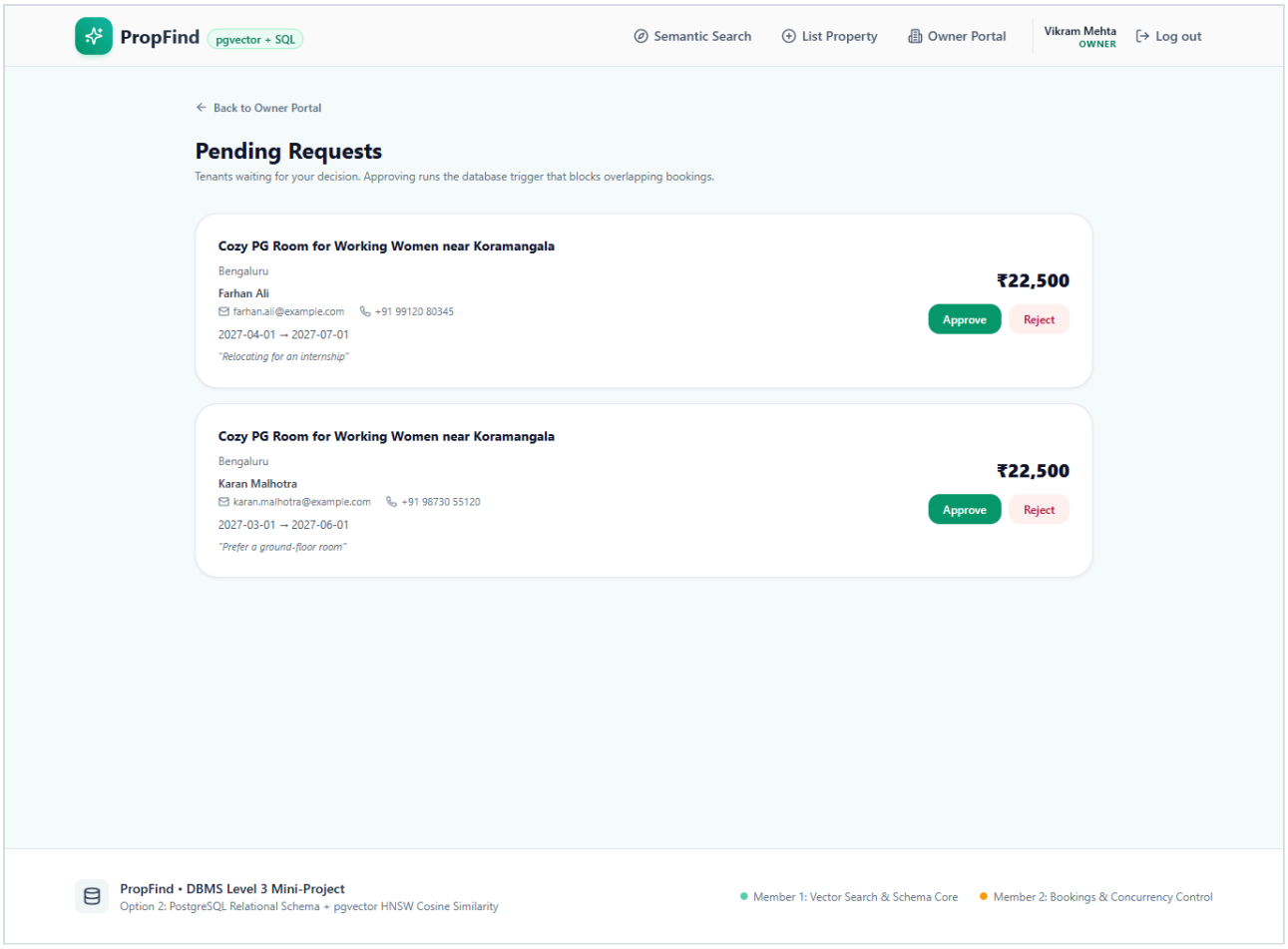


Figure 8.15: pending requests

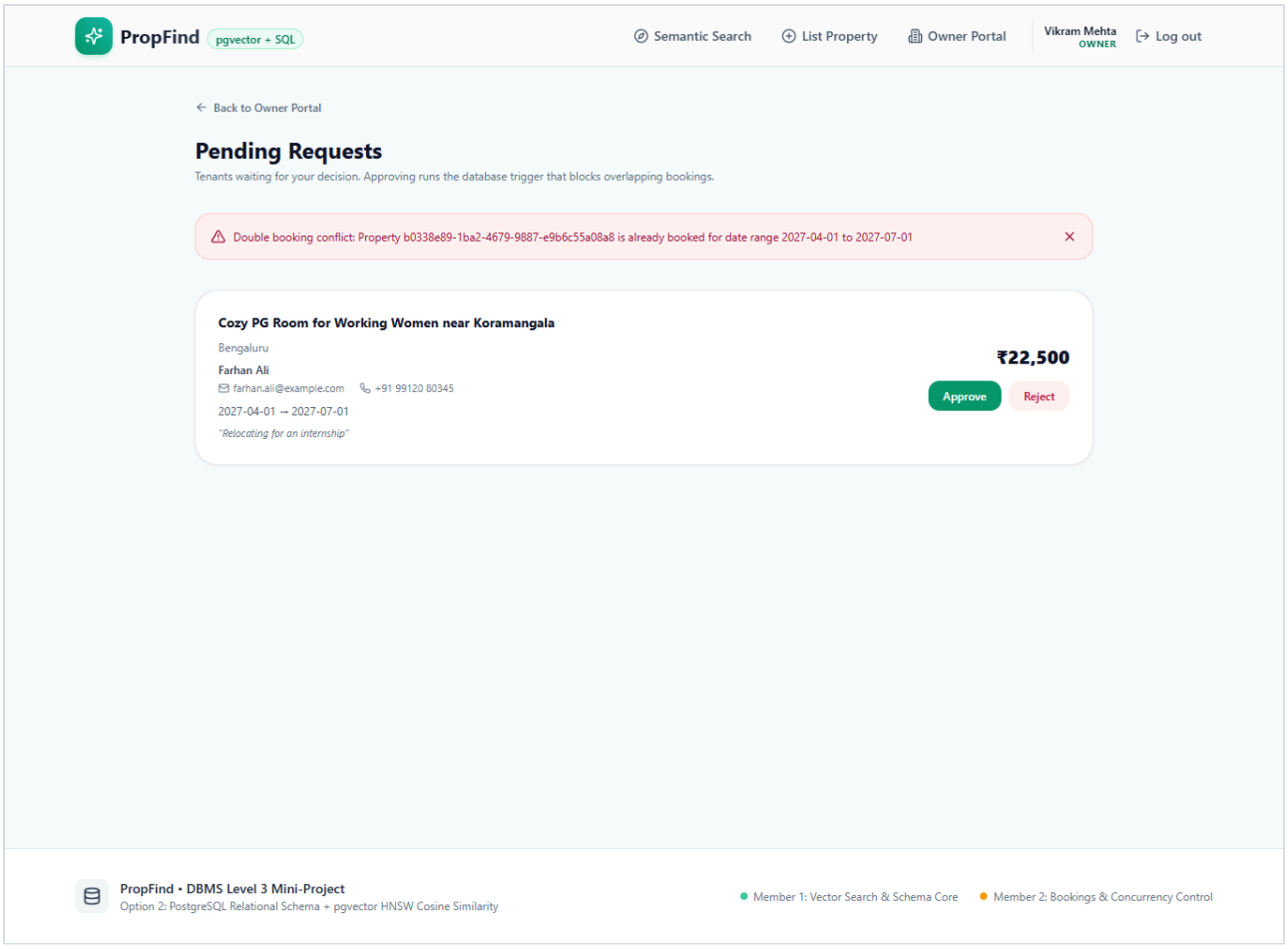


Figure 8.16: approving an overlapping booking is rejected by trg_validate_booking_overlap and the database message is shown

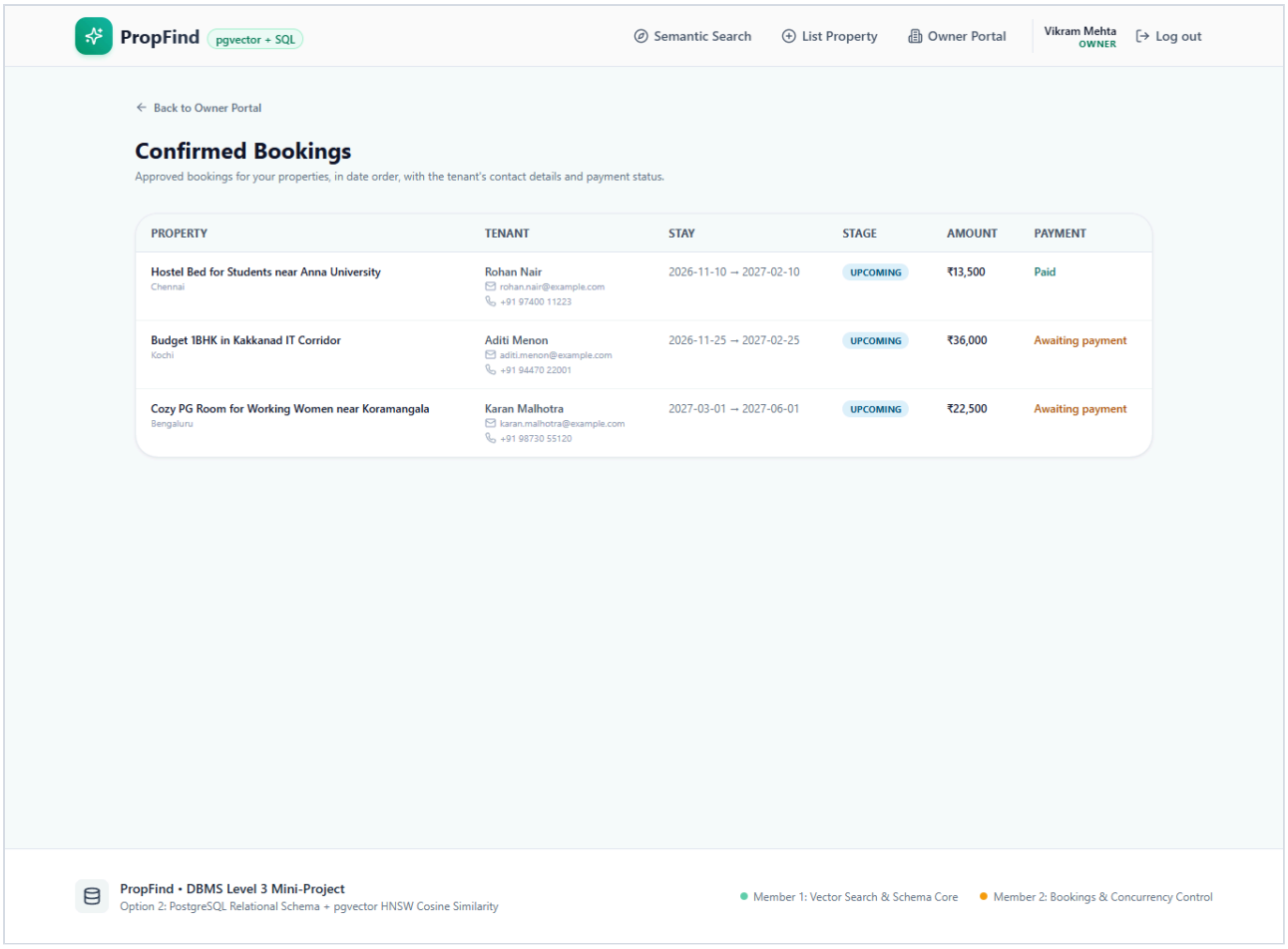


Figure 8.17: confirmed bookings

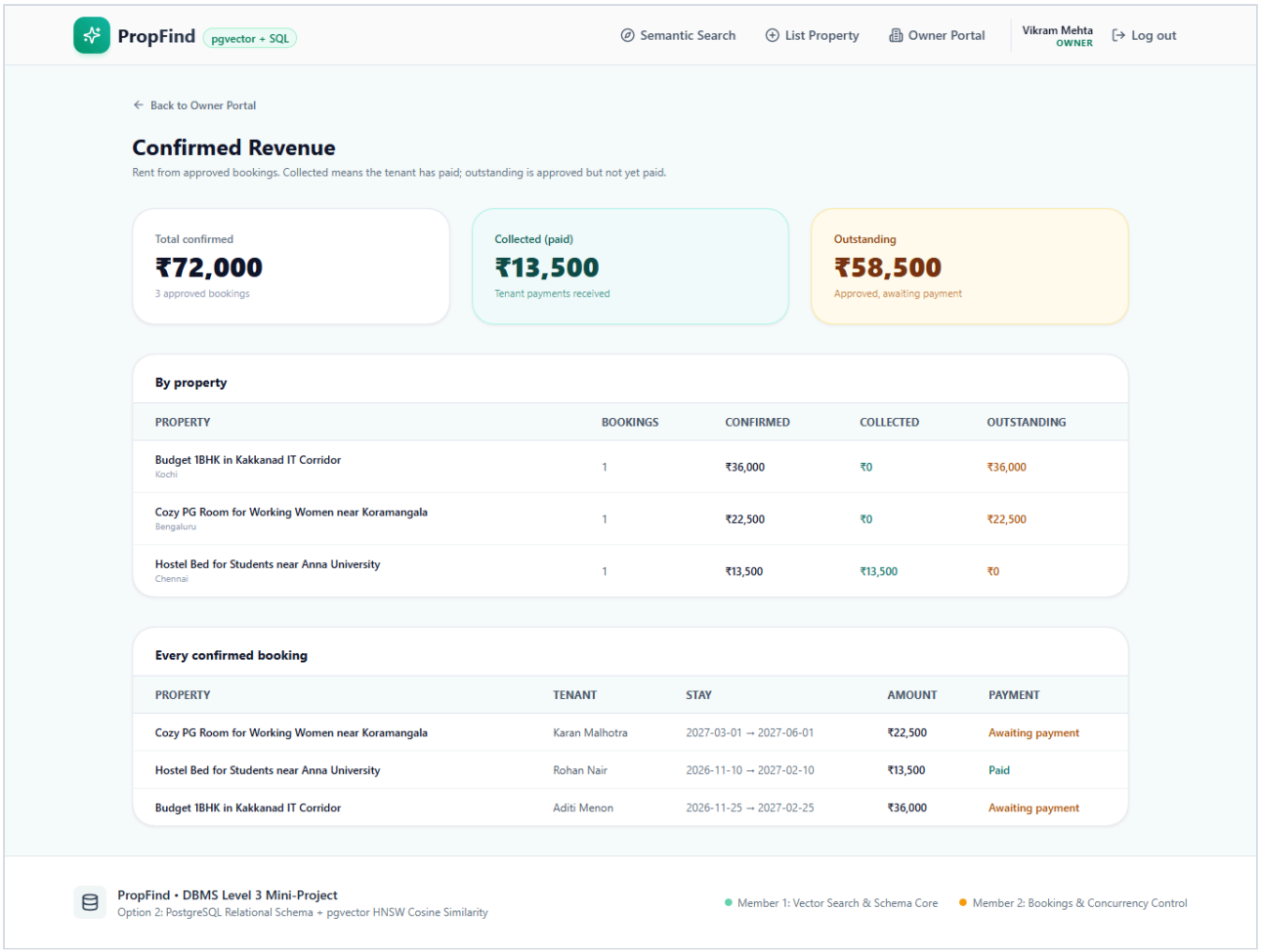


Figure 8.18: confirmed revenue: collected vs outstanding, per property

9. Evaluation Rubrics – Completion Status

Status key: **Done** = implemented and demonstrated; **Partial** = implemented with a stated limitation; **Not done** = out of scope.

Requirement (from the project brief)	Status	Evidence / note
Two-member team, defined responsibilities, both database and application work	Done	Section 2; each member has SQL and application commits
Option 2: SQL (PostgreSQL) + vector database (pgvector)	Done	Sections 5 and 6
ER diagram converted to relational schema; entities, attributes, PK, FK, relationships, constraints	Done	Figure 5.1, relational schema in 5.1, data dictionary in the appendix
Vector data model documented: embeddings, metadata, indexes, retrieval strategy	Done	Section 6.1 table
DDL: CREATE, ALTER, DROP, keys, constraints, indexes, views	Done	5.2, 5.3 (ALTER/DROP demos in db/queries.sql), 5.5
DML: INSERT, UPDATE, DELETE, SELECT	Done	Seed script, 5.4, application services
JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries	Done	Figures 5.8 to 5.10 (30+ labelled queries)
At least four advanced concepts	Done	Transactions, views, stored functions, 7 triggers, indexing, vector similarity search (6 concepts)
Full-text search	Partial	Shown as an ad-hoc tsvector query (Figure 6.3); no dedicated GIN index or UI
Explain embeddings: what, which model, storage, metric, retrieval	Done	Section 6.1
Functional Next.js interface with forms, tables, search, filtering, dashboards	Done	Section 8
Backend/API layer between Next.js and the database	Done	Route handlers + service layer (Figure 4.1)
No hard-coded credentials; environment variables	Done	.env / .env.example; app fails to start without DATABASE_URL. The seed script has a documented demo-password fallback (password123) for demo accounts only
Authentication and at least 2 roles	Done	Owner and tenant (self-register) with role checks in middleware and in every API route; admin account exists
Admin features	Partial	Admin role is seeded and passes owner checks, but there is no dedicated admin screen
Validation at the database level	Done	ENUMs, CHECK, UNIQUE, FK, triggers; 22 negative tests (Figure 5.20)

Requirement (from the project brief)	Status	Evidence / note
Security basics: hashed passwords, no plain-text secrets, safe queries	Done	bcrypt, httpOnly JWT cookie, parameterised SQL everywhere. Not done: rate limiting, CSRF tokens (SameSite=Lax only)
Documentation: problem, objectives, scope, ER, schema, DB choice, dictionary, queries, architecture, screenshots, tests, conclusion	Done	This report plus README and docs/
Git/GitHub with individual contributions and README with setup	Done	Feature branches and pull requests; 54 commits (Kanishq) and 2 commits (Jishnu) excluding merges; README has setup steps
Testing results	Partial	SQL: 23/23 checks pass; parser: 12/12 pass; API flows tested with scripted requests. No automated browser (UI) tests
Originality; goes beyond CRUD	Done	Semantic + hybrid search, concurrency control, trigger-enforced state machine, audit log, refunds
Hosted database (optional)	Prepared	DATABASE_SSL option and docs/HOSTED_DATABASE.md for Supabase / Neon; demo database runs locally in Docker

Testing results

Test	How to run	Result
Database integrity and transactions (23 checks)	psql -f db/transactions_tests.sql	23 passed, 0 failed
Labelled SQL demonstrations	psql -f db/queries.sql	Runs with ON_ERROR_STOP, exit code 0; data unchanged afterwards
Natural-language parser (12 cases)	npm run test:parser	12 / 12 passed
Production build	npm run build	Passes (13 routes)
End-to-end flow (scripted)	register, login, search, book, approve, pay, double-booking rejection, role redirects	All behaved as expected

Appendix A: Data Dictionary

Generated from `information_schema` on the running database (PostgreSQL 16.15, pgvector 0.8.7). Current data: 14 users, 59 properties in 15 cities, 44 reviews, 14 bookings, 3 payments, 22 audit rows.

booking_audit

Column	Type	Null	Default
id	bigint	NOT NULL	nextval('booking_audit_id_seq'::regcla
booking_id	uuid	NOT NULL	
old_status	booking_status	NULL	
new_status	booking_status	NOT NULL	
changed_by	uuid	NULL	
changed_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

bookings

Column	Type	Null	Default
id	uuid	NOT NULL	uuid_generate_v4()
property_id	uuid	NOT NULL	
tenant_id	uuid	NOT NULL	
start_date	date	NOT NULL	
end_date	date	NOT NULL	
total_amount	numeric(10,2)	NOT NULL	
status	booking_status	NOT NULL	'PENDING'::booking_status
special_requests	text	NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP
updated_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

payments

Column	Type	Null	Default
id	uuid	NOT NULL	uuid_generate_v4()
booking_id	uuid	NOT NULL	
amount	numeric(10,2)	NOT NULL	
method	payment_method	NOT NULL	
status	payment_status	NOT NULL	'PAID'::payment_status

Column	Type	Null	Default
transaction_ref	character varying(40)	NOT NULL	('TXN-'::text
paid_at	timestamp with time zone	NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

properties

Column	Type	Null	Default
id	uuid	NOT NULL	uuid_generate_v4()
owner_id	uuid	NOT NULL	
title	character varying(200)	NOT NULL	
description	text	NOT NULL	
property_type	character varying(50)	NOT NULL	'Apartment'::character varying
price_per_month	numeric(10,2)	NOT NULL	
security_deposit	numeric(10,2)	NOT NULL	0.00
bedrooms	integer	NOT NULL	
bathrooms	integer	NOT NULL	
area_sqft	integer	NOT NULL	
address	text	NOT NULL	
city	character varying(80)	NOT NULL	
state	character varying(80)	NOT NULL	
pincode	character varying(20)	NOT NULL	
status	property_status	NOT NULL	'AVAILABLE'::property_status
amenities	text[]	NULL	'{}'::text[]
image_url	text	NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP
updated_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

property_embeddings

Column	Type	Null	Default
property_id	uuid	NOT NULL	
embedding	vector	NOT NULL	
model_name	character varying(100)	NOT NULL	'all-MiniLM-L6-v2'::character varying
indexed_content	text	NOT NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

Column	Type	Null	Default
updated_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

reviews

Column	Type	Null	Default
id	uuid	NOT NULL	uuid_generate_v4()
property_id	uuid	NOT NULL	
tenant_id	uuid	NOT NULL	
rating	integer	NOT NULL	
comment	text	NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

users

Column	Type	Null	Default
id	uuid	NOT NULL	uuid_generate_v4()
full_name	character varying(120)	NOT NULL	
email	character varying(160)	NOT NULL	
password_hash	character varying(255)	NOT NULL	
role	user_role	NOT NULL	'TENANT'::user_role
phone	character varying(20)	NULL	
avatar_url	text	NULL	
created_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP
updated_at	timestamp with time zone	NOT NULL	CURRENT_TIMESTAMP

Constraints

Table	Constraint	Type	Definition
users	users_email_format	CHECK	CHECK (((email)::text ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'::text))
users	users_phone_format	CHECK	CHECK (((phone IS NULL) OR ((phone)::text ~ '^\+?[0-9]{10,20}$'::text)))
users	users_pkey	PRIMARY KEY	PRIMARY KEY (id)
users	users_email_key	UNIQUE	UNIQUE (email)
properties	properties_area_sqft_check	CHECK	CHECK ((area_sqft > 0))

Table	Constraint	Type	Definition
properties	properties_bathrooms_check	CHECK	CHECK ((bathrooms >= 0))
properties	properties_bedrooms_check	CHECK	CHECK ((bedrooms >= 0))
properties	properties_price_per_month_check	CHECK	CHECK ((price_per_month > (0)::numeric))
properties	properties_security_deposit_check	CHECK	CHECK ((security_deposit >= (0)::numeric))
properties	properties_owner_id_fkey	FOREIGN KEY	FOREIGN KEY (owner_id) REFERENCES users(id) ON DELETE CASCADE
properties	properties_pkey	PRIMARY KEY	PRIMARY KEY (id)
property_embeddings	property_embeddings_property_id_fkey	FOREIGN KEY	FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE CASCADE
property_embeddings	property_embeddings_pkey	PRIMARY KEY	PRIMARY KEY (property_id)
bookings	bookings_total_amount_check	CHECK	CHECK ((total_amount >= (0)::numeric))
bookings	check_booking_dates	CHECK	CHECK ((end_date > start_date))
bookings	bookings_property_id_fkey	FOREIGN KEY	FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE RESTRICT
bookings	bookings_tenant_id_fkey	FOREIGN KEY	FOREIGN KEY (tenant_id) REFERENCES users(id) ON DELETE RESTRICT
bookings	bookings_pkey	PRIMARY KEY	PRIMARY KEY (id)
reviews	reviews_rating_check	CHECK	CHECK (((rating >= 1) AND (rating <= 5)))
reviews	reviews_property_id_fkey	FOREIGN KEY	FOREIGN KEY (property_id) REFERENCES properties(id) ON DELETE CASCADE
reviews	reviews_tenant_id_fkey	FOREIGN KEY	FOREIGN KEY (tenant_id) REFERENCES users(id) ON DELETE CASCADE
reviews	reviews_pkey	PRIMARY KEY	PRIMARY KEY (id)
reviews	unique_tenant_property_review	UNIQUE	UNIQUE (property_id, tenant_id)
payments	payments_amount_check	CHECK	CHECK ((amount > (0)::numeric))

Table	Constraint	Type	Definition
payments	payments_paid_has_timestamp	CHECK	CHECK (((status <> 'PAID'::payment_status) OR (paid_at IS NOT NULL)))
payments	payments_booking_id_fkey	FOREIGN KEY	FOREIGN KEY (booking_id) REFERENCES bookings(id) ON DELETE RESTRICT
payments	payments_pkey	PRIMARY KEY	PRIMARY KEY (id)
payments	payments_transaction_ref_key	UNIQUE	UNIQUE (transaction_ref)
booking_audit	booking_audit_pkey	PRIMARY KEY	PRIMARY KEY (id)

Appendix B: Individual Contributions and History

Member	Commits (excl. merges)	Lines added	Lines removed
KANISHQ SURENDRAN (PES2UG24CS214)	54	3,654	978
JISHNU SIVAKUMAR (PES2UG24CS204)	2	4,120	40

Measured with `git log --numstat` on the main branch, excluding `package-lock.json`. Work was merged through pull requests from separate feature branches (authentication, booking life cycle, payments and audit, real embeddings, SQL demonstrations, dataset and search, owner dashboard pages, hosted database support, project report), so history shows who did what.

Appendix C: Limitations and Future Work

- No payment gateway: a payment is recorded (UPI / card / net banking label) but no money moves.
- No admin screen, no property editing or deletion UI, no image upload (images are URLs).
- Search quality depends on a small embedding model and a 58-listing dataset; some phrasings (for example “apartment that allows pets”) rank less cleanly than others.
- Full-text search has no dedicated GIN index; an availability calendar, lease extensions and saved listings are possible extensions.
- No automated browser tests; rate limiting and CSRF tokens would be added for production.

Conclusion

PropFind shows how one PostgreSQL database can serve both paradigms required by the project: a relational core whose integrity is guaranteed by constraints, transactions and triggers, and a vector layer that makes search match meaning rather than keywords. The most important design decision was keeping hard numeric constraints in SQL and using vectors only for what vectors are good at. The result is a working Next.js application whose behaviour is backed by queries and tests that can be re-run from the repository.